

자바 프로그래밍 v21

inky4832@daum.net

1장. 자바 개요 및 개발환경

1. 자바 프로그래밍 개요
2. 자바 특징
3. JVM (Java Virtual Machine)
4. JDK 21 설치
5. IntelliJ IDEA 설치
6. 기본 코드 구성
7. 자바 API 문서

1. 자바 프로그래밍 개요

JDK 21

- Sun Microsystems에서 1996년에 개발된 객체 지향 프로그래밍 언어.
- 현재는 오라클에서 라이선스(License) 관리.

<https://www.tiobe.com/tiobe-index>

Sep 2026	Sep 2025	Change	Programming Language		Ratings	Change
1	1			Python	17.76%	-8.22%
2	3	▲		C	10.28%	+1.63%
3	2	▼		C++	8.67%	-0.13%
4	4			Java	7.54%	-0.81%
5	5			C#	4.22%	-2.16%
6	6			JavaScript	2.76%	-0.46%

2. 버전에 따른 지원 현황

JDK 21

<https://www.oracle.com/java/technologies/java-se-support-roadmap.html>

Oracle Java SE Support Roadmap*†				
Release	GA Date	Premier Support Until	Extended Support Until	Sustaining Support
8 (LTS)**	March 2014	March 2022	December 2030*****	Indefinite
9 - 10 (non-LTS)	September 2017 - March 2018	March 2018 - September 2018	Not Available	Indefinite
11 (LTS)	September 2018	September 2023	January 2032*****	Indefinite
12 - 16 (non-LTS)	March 2019 - March 2021	September 2019 - September 2021	Not Available	Indefinite
17 (LTS)	September 2021	September 2026****	September 2029****	Indefinite
18 - 20 (non-LTS)	March 2022 - March 2023	September 2022 - September 2023	Not Available	Indefinite
21 (LTS)	September 2023	September 2028****	September 2031****	Indefinite
22 - 24 (non-LTS)	March 2024 - March 2025	September 2024 - September 2025	Not Available	Indefinite
25 (LTS)	September 2025	September 2030****	September 2033****	Indefinite
26 (non-LTS)***	March 2026	September 2026	Not Available	Indefinite
27 (non-LTS)***	September 2026	March 2027	Not Available	Indefinite

○ 특징

- 운영체제 독립적 (Write once, Run anywhere)
- 객체지향 언어 (Object Oriented Programming)
- 다중 스레드 지원 (multi thread)
- 간단한 코드 작성 (포인터 제거 및 Garbage Collection 활용)
- 다양한 외부 API (Open Source library 지원)

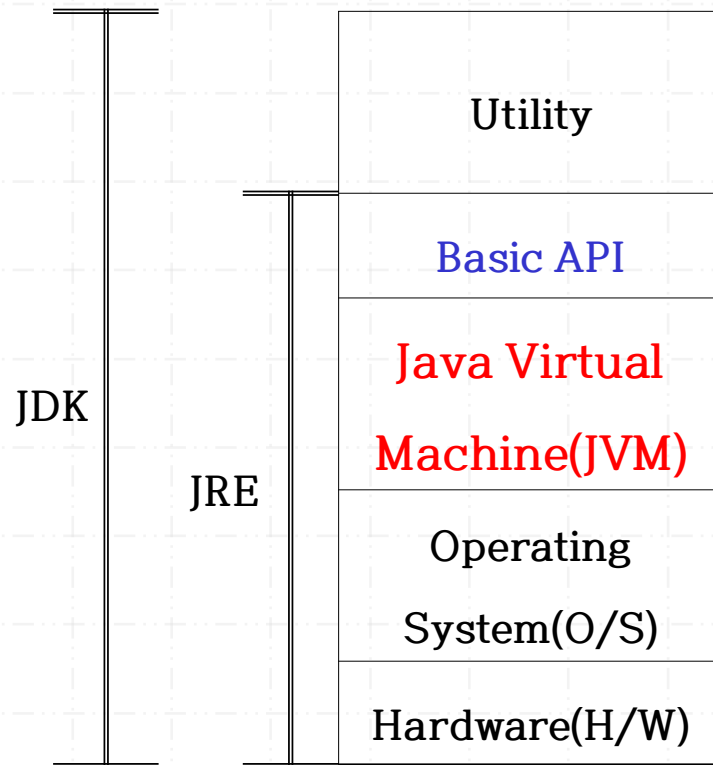
○ Garbage Collector

- 프로그램에서 더 이상 사용하지 않는 메모리 공간을 자동으로 모아서 처리하는 프로세스.
- 시스템이 한가하거나 메모리가 부족한 위급 상황에서 GC가 동작.
- `System.gc();` 사용하여 수행 권유 가능.

3. JVM (Java Virtual Machine)

JDK 21

- 다양한 운영체제에서 동작할 수 있도록 자바가상머신(JVM: Java Virtual Machine)이 제공된다.



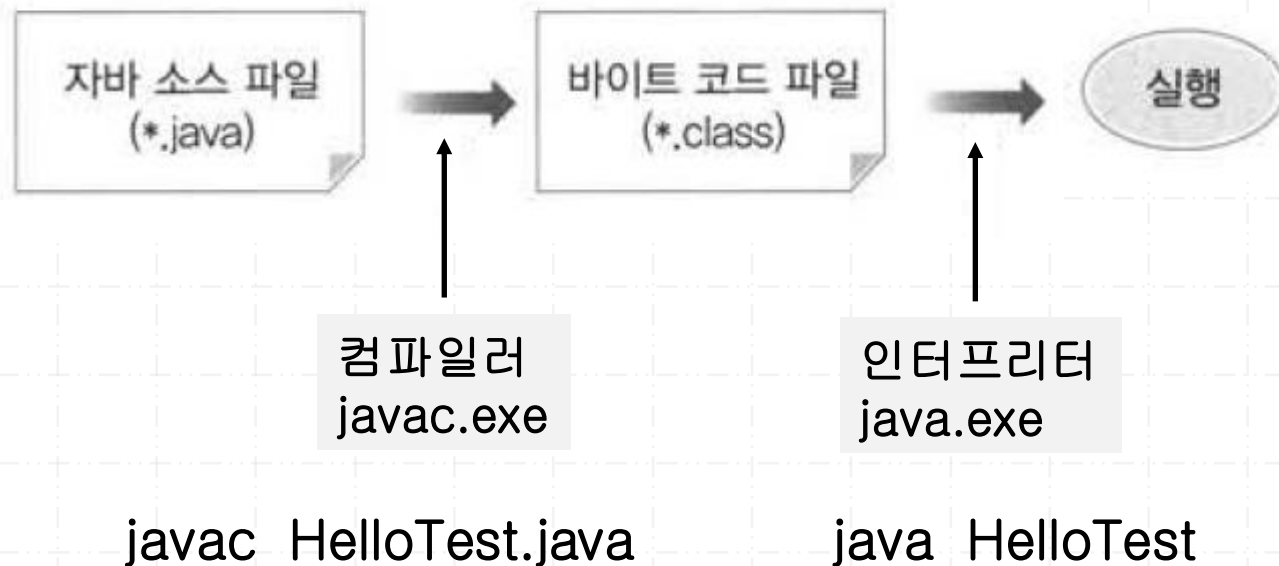
JDK (Java Development Kit)

JRE (Java Runtime Environment)

3. JVM (Java Virtual Machine)

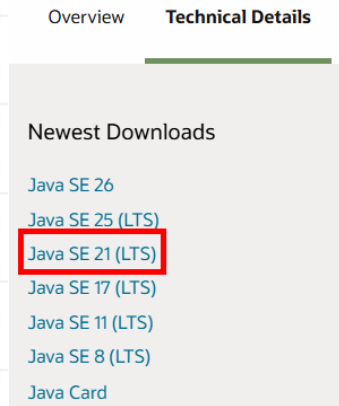
JDK 21

- 실제 하드웨어에 소프트웨어가 설치되어 구현된 가상 CPU이다.
- 내부적으로 여러 가지 하드웨어 특징을 갖는 구조를 제공한다.
(stack, heap, Register Set 등)
- 플랫폼에 독립적으로 컴파일 된 바이트코드(bytecode, 클래스파일)를 실행한다.
- JDK를 설치하거나 또는 브라우저에서 제공된다.



가. 프로그램 다운로드 (회원가입 및 로그인 필수)

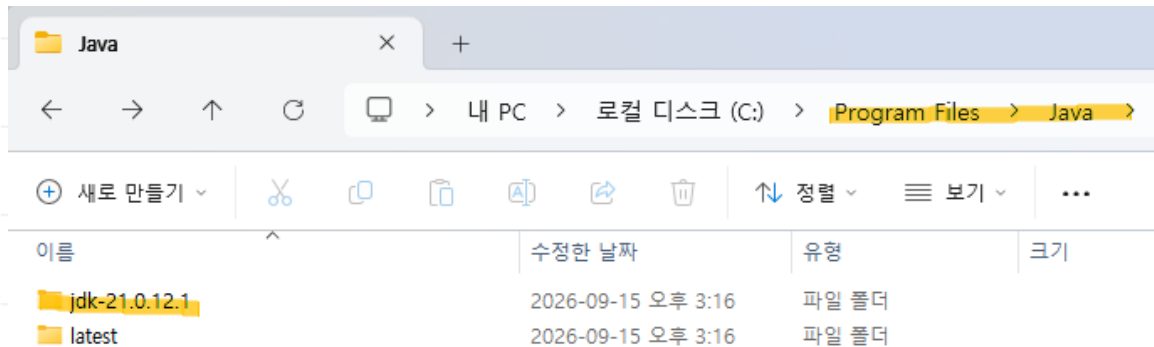
<https://www.oracle.com/java/technologies/>



버전은 업데이트 상황에 따라서 변경될 수 있음.

Linux	macOS	Windows
Product/file description	File size	Download
x64 Compressed Archive	187.82 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.zip (sha256)
x64 Installer	166.94 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.exe (sha256)
x64 MSI Installer	165.71 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.msi (sha256)

나. 설치 시작 (기본 설치 권장)



기본적으로 c:\Program Files\Java 폴더에 설치된다.

다. 환경 변수 설정

JAVA_HOME=C:\Program Files\Java\jdk-21.0.12.1

PATH=%JAVA_HOME%\bin;%PATH%

```
C:\Users\User>set JAVA_HOME
JAVA_HOME=C:\Program Files\Java\jdk-21.0.12.1

C:\Users\User>set PATH
Path=C:\Program Files\Java\jdk-21.0.12.1\bin;C:\Program Fi
```

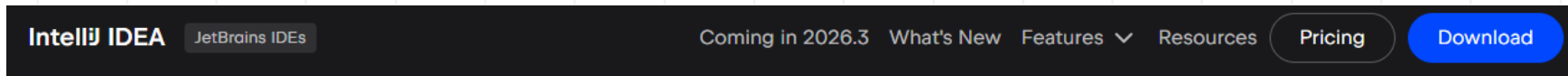
```
C:\Users\User>java -version
java version "21.0.12.1" 2026-08-18 LTS
Java(TM) SE Runtime Environment (build 21.0.
Java HotSpot(TM) 64-Bit Server VM (build 21.
```

5. IntelliJ IDEA 설치

JDK 21

가. 프로그램 다운로드

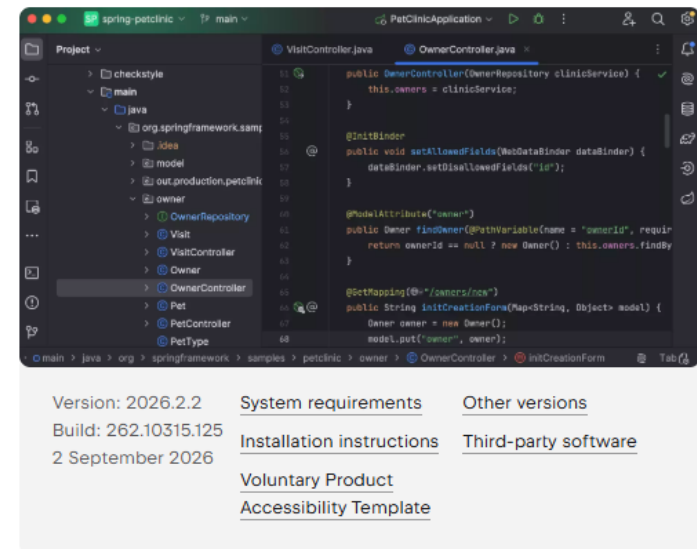
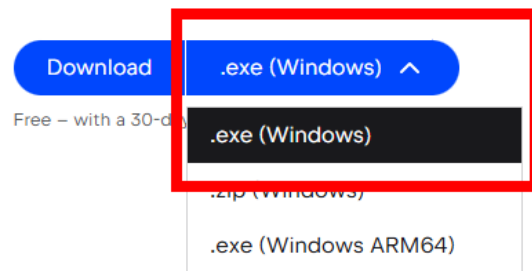
<https://www.jetbrains.com/idea>



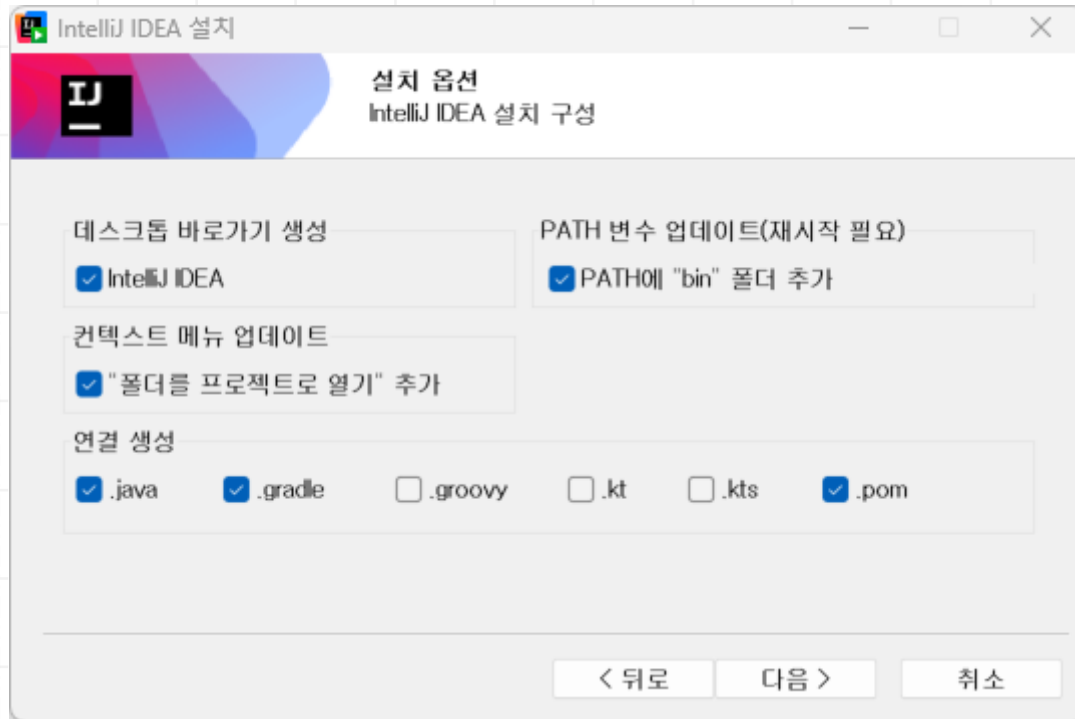
Windows macOS Linux



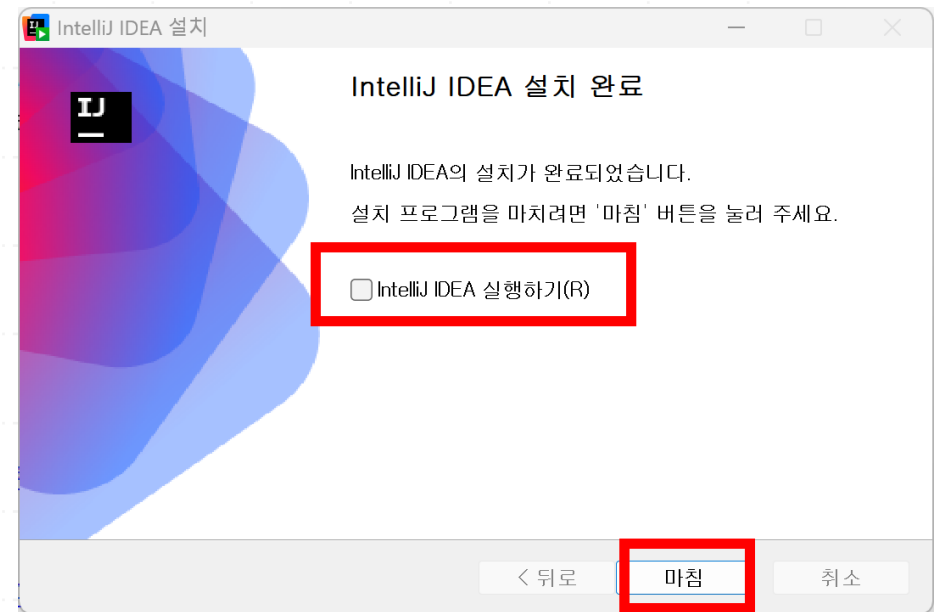
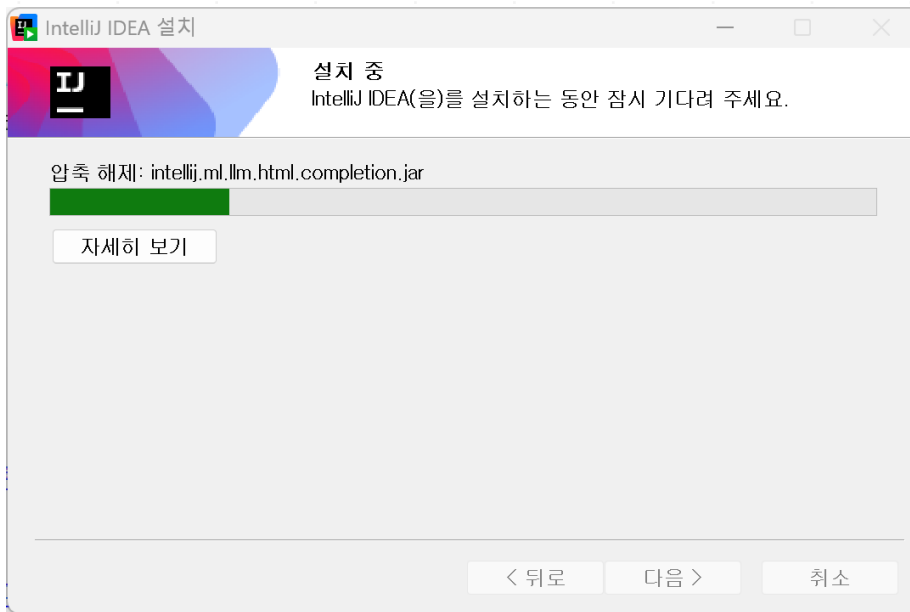
The Leading IDE for Professional Development in Java and Kotlin



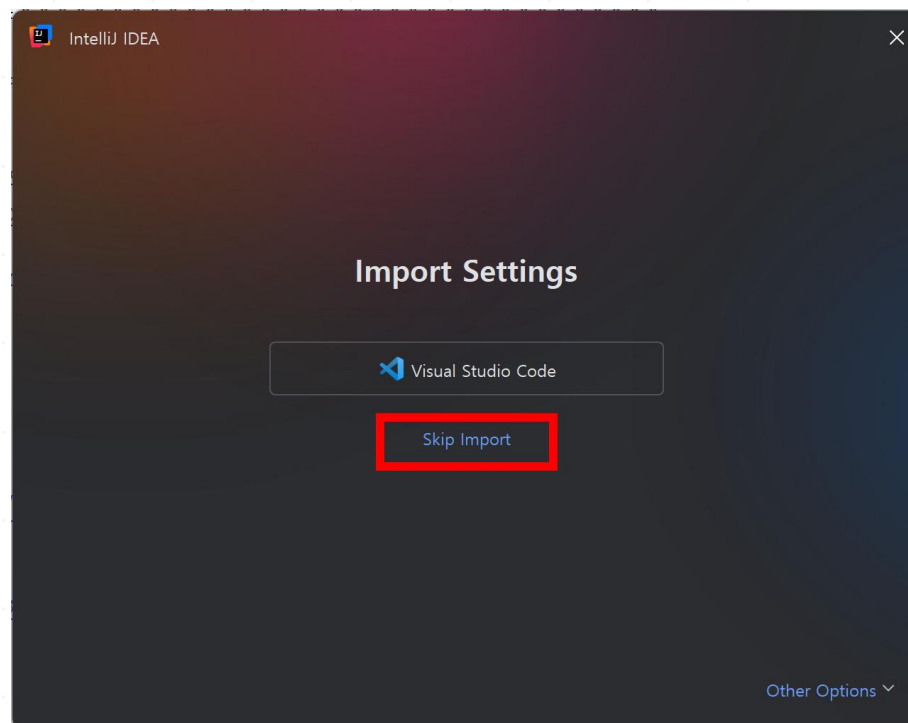
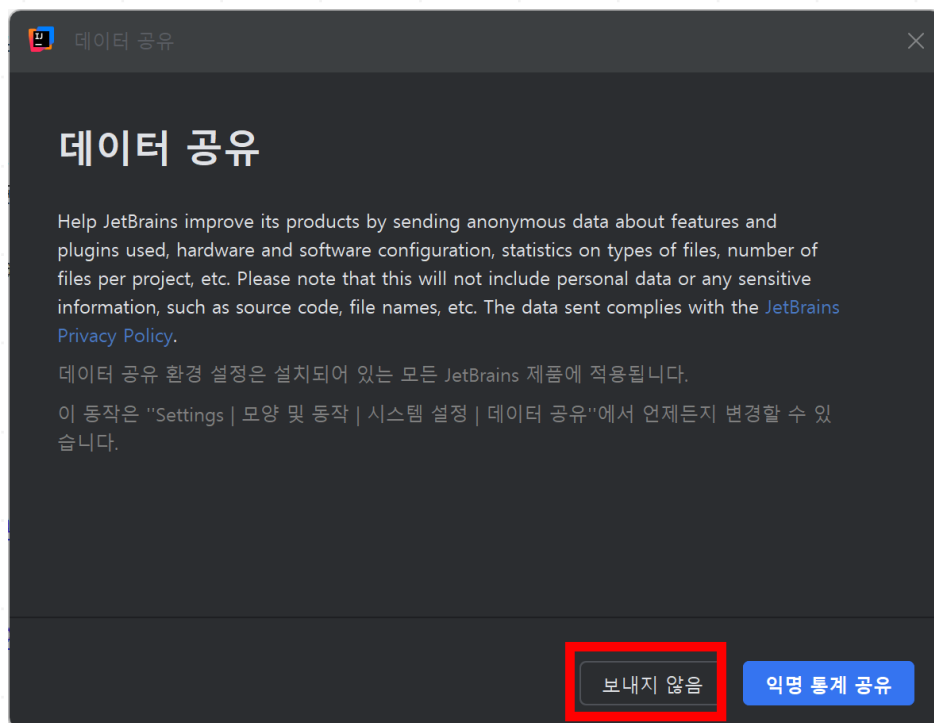
나. IntelliJ IDEA 설치



나. IntelliJ IDEA 설치

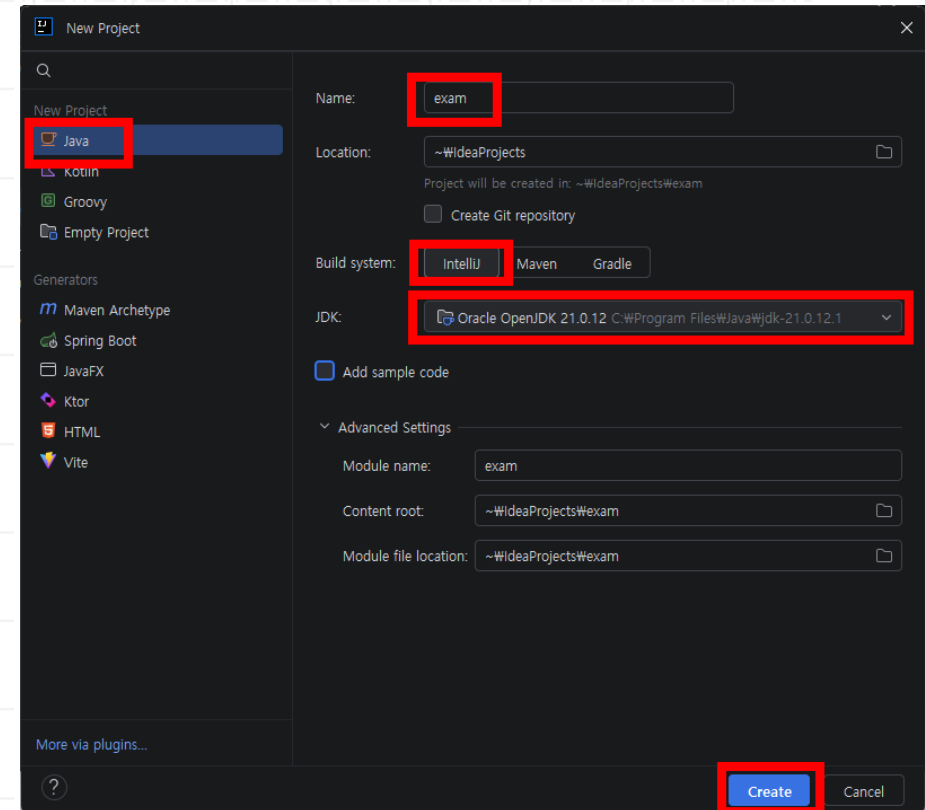
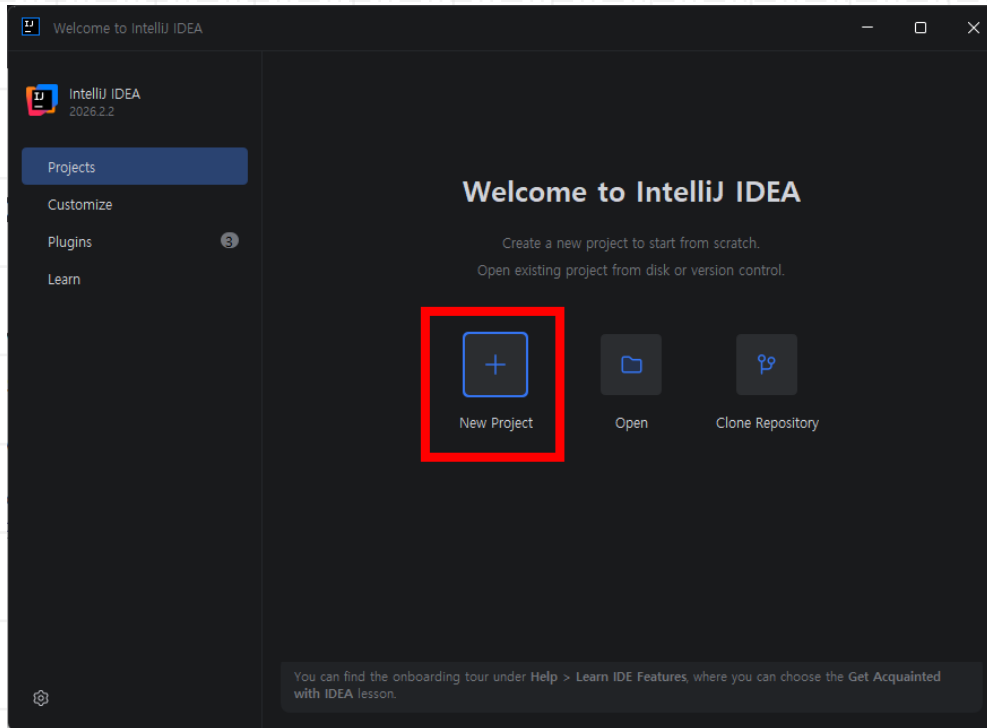


다. IntelliJ IDEA 실행



5. IntelliJ IDEA 설치

JDK 21



라. 기본 코드 작성

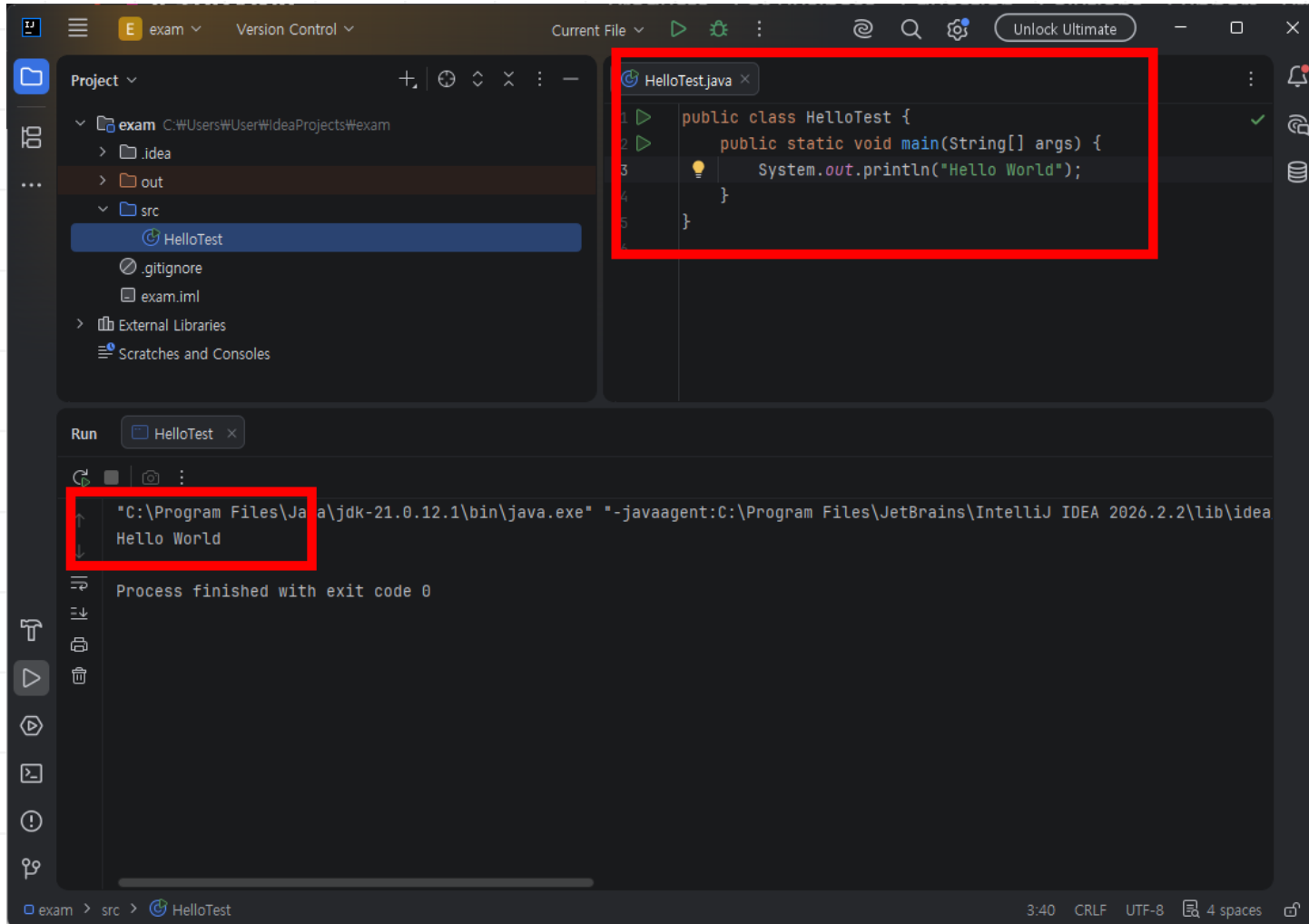
main 메서드 정의 시 다음 보기와 같이 대/소문자 구분하여 정확히 입력할 것

클래스정의	public class HelloTest{
메서드정의	public static void main(String[] args){ System.out.println("Hello World"); }
	}

6. 기본 코드 구성

JDK 21

마. 기본 코드 실행 및 콘솔 출력



○ 자바 API 문서

- 이미 제공되는 클래스들(API)에 대한 사용 방법을 문서화 하여 제공해 주는 도움말과 같은 것이다.

<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/module-summary.html>

Exports	
Package	Description
<code>java.io</code>	Provides for system input and output through data streams, serialization and the file system.
<code>java.lang</code>	Provides classes that are fundamental to the design of the Java programming language.
<code>java.lang.annotation</code>	Provides library support for the Java programming language annotation facility.
<code>java.lang.invoke</code>	The <code>java.lang.invoke</code> package provides low-level primitives for interacting with the Java Virtual Machine.
<code>java.lang.module</code>	Classes to support module descriptors and creating configurations of modules by means of resolution and service binding.
<code>java.lang.ref</code>	Provides reference-object classes, which support a limited degree of interaction with the garbage collector.
<code>java.lang.reflect</code>	Provides classes and interfaces for obtaining reflective information about classes and objects.
<code>java.math</code>	Provides classes for performing arbitrary-precision integer arithmetic (<code>BigInteger</code>) and arbitrary-precision decimal arithmetic (<code>BigDecimal</code>).
<code>java.net</code>	Provides the classes for implementing networking applications.

2장. 식별자와 데이터형

1. 식별자(identifier)
2. 데이터형(Data Type)
3. 리터럴(literal)
4. 변수 (variable)
5. 데이터형 변환
6. 상수

1. 식별자(identifier)

JDK 21

- 식별자는 자바코드 내에서 사용되는 개별적인 **단어**를 의미한다.
- 식별자 지정 규칙
 - 첫 문자는 반드시 영문자만 가능.
그 다음 문자부터는 숫자와 문자를 혼합해서 사용 가능.
_ 와 \$는 첫 문자로 사용할 수 있는 특수문자.
 - 식별자에 대한 길이 제한은 없다.
 - 자바는 대소문자를 구별한다.
- 식별자 종류 2가지
 - a. 시스템 정의 식별자
자바 시스템이 필요에 의해서 먼저 정의한 식별자로서 보통 '예약어', '키워드'라고 부른다.
 - b. 사용자 정의 식별자
자바 개발자가 필요에 의해서 정의한 식별자로서 **클래스명, 변수명, 메서드명**을 지정할 때 사용된다.

1. 식별자(identifier)

JDK 21

○ 시스템 정의 식별자 (예약어, 키워드) 종류

- 자바언어 자체에서 특별한 의미를 가지는 식별자이다.
- 시스템이 먼저 지정한 식별자이기 때문에 사용자 지정 식별자로서 사용이 불가.

분류	예약어
기본 데이터 타입	boolean, byte, char, short, int, long, float, double
접근 지정자	private, protected, public
클래스와 관련된 것	class, abstract, interface, extends, implements, enum
객체와 관련된 것	new, instanceof, this, super, null
메소드와 관련된 것	void, return
제어문과 관련된 것	if, else, switch, case, default, for, do, while, break, continue
논리값	true, false
예외 처리와 관련된 것	try, catch, finally, throw, throws
기타	transient, volatile, package, import, synchronized, native, final, static, strictfp, assert

1. 식별자(identifier)

JDK 21

○ 사용자 정의 식별자

사용자가 지정 가능한 식별자로서 클래스명, 변수명, 메서드명, 상수 사용시 이름을 지정할 때 사용된다.

구분	정의 규칙	사용 예
클래스	□ 첫 문자는 항상 대문자로 표현 □ 하나 이상의 단어가 합쳐질 때는 각 단어의 첫 문자들만 대문자로 표현 (Camel 표기법) □ 의미 있는 명사형으로 지정.	<pre>class Student{ ...; }</pre>
변수와 메서드	□ 첫 문자는 항상 소문자로 표현 □ 하나 이상의 단어가 합쳐질 때는 두 번째부터 오는 단어의 첫 문자들만 대문자로 표현 (Camel 표기법) □ 변수는 의미 있는 명사형으로 메서드는 의미 있는 동사형으로 지정.	<pre>String custId; public String getName(){ ...; }</pre>
상수	□ 모든 문자를 대문자로 표현 □ 하나 이상의 단어가 합쳐질 때 공백 필요 시 under score(_)를 사용하여 연결한다. □ 의미 있는 명사형으로 지정.	<pre>int javaTest= 10; final int JAVA_TEST = 20;</pre>

○ 데이터형은 자바언어가 처리할 수 있는 데이터 종류를 의미한다.

○ 자바의 데이터형 (Data Type) 2가지

- 기본 데이터형 (primitive data type : PDT)

- 수치형(정수형): byte ,short , int , long
- 수치형(실수형): float, double
- 논리형: boolean
- 수치형(문자형): char

- 참조 데이터형 (reference data type : RDT)

- 기본 데이터형을 제외한 나머지 데이터형.
- 대표적으로 **클래스** , **배열** , **인터페이스**가 있다.

○ 기본 자료형의 종류

[표 2-5] 기본 자료형의 종류

자료형	키워드	크기	기본값	표현 범위
논리형	boolean	1byte	false	true 또는 false(0과 1이 아니다)
문자형	char	2byte	\u0000	0 ~ 65,535
정수형	byte	1byte	0	-128 ~ 127
	short	2byte	0	-32,768 ~ 32,767
	int	4byte	0	-2,147,483,648 ~ 2,147,483,647
	long	8byte	0	-9,223,372,036,854,775,808 ~ 9,223,372,036,854,775,807
실수형	float	4byte	0.0	-3.4E38 ~ +3.4E38
	double	8byte	0.0	-1.7E308 ~ +1.7E308

○ 정수형의 기본은 int 이고 실수형의 기본은 double 이다.

3. 리터럴(literal)

JDK 21

- 리터럴(literal)은 자바 언어가 처리하는 실제 값을 의미한다.

1) 문자

하나의 문자를 의미한다. 반드시 "(single quotes)으로 표현한다.

예> 'A' , '남'

다음은 특별한 형태의 escape 문자이다.

이스케이프 문자	용도	유니코드
\t	수평 탭	0x0009
\n	줄 바꿈	0x000a
\r	리턴	0x000d
\"	" (큰따옴표)	0x0022
\'	' (작은따옴표)	0x0027
\\	\	0x005c

- 문자열은 반드시 ""(double quotes)으로 표현한다. (문자와 문자열은 구분됨)

예> "홍길동" , "서울시"

2) 정수

일반적인 정수 데이터를 의미한다. 10진수, 2진수, 8진수, 16진수로 표현이 가능하다. 8진수는 숫자 0부터 7까지의 숫자데이터 조합으로 표현 가능하고, 16진수는 숫자 0과 x의 뒤에 숫자 0부터 9까지, 문자 A부터 F까지의 조합으로 표현한다.

예> 234 //10진수
 030 // 8진수
 0xA //16진수

int 보다 큰 값은 long 타입으로 표현하기 위해서 값 마지막에 L 문자를 추가한다.

예> long ssn = 8012101234567L;

3) 실수

일반적인 소수점을 가진 실수 데이터를 의미한다.

```
예> 3.14      // 일반적인 표현방식으로서 기본적으로 double로 처리
     6.02E23   // 지수표현방식으로 큰 실수데이터 표현시 사용
     2.71F     // 간단한 float 형을 표현
```

실수형의 기본은 double이기 때문에 float을 표현하기 위해서는 3.14F 또는 3.14f를 사용하며 명시적 double 표현법은 3.14D 또는 3.14d 형식을 사용한다.

4) 논리

참/거짓을 표현할 때 사용하는 논리 데이터이다.
자바에서는 소문자 true 또는 false 로 논리값을 표현한다.

```
예> true
     false
```

개요

프로그램에서 사용하는 데이터(리터럴)를 저장하기 위한 용도로 사용된다.

복수개의 값이 아닌 단 하나의 값만 저장이 가능하다.

복수개의 값을 저장하기 위해서 배열 또는 컬렉션을 사용한다.

변수에는 다양한 타입의 값을 저장하지 못하고 한가지 타입만 저장 가능하다.

저장된 데이터는 언제든지 변경이 가능하기 때문에 '변경이 가능한 수'

즉, 변수라고 부른다. 변경이 불가능한 수는 '상수'라고 부른다.

기본형 데이터를 저장하면 '기본형 변수' 라고 하고 참조형 데이터를 저장하면 '참조형 변수'라고 한다.

다음과 같은 순서로 변수를 사용할 수 있다.

- 1) 변수 선언
- 2) 값 할당(초기화)
- 3) 값 변경

4. 변수 (variable)

JDK 21

1) 변수 선언

변수 선언은 자바 프로그램에게 저장 시킬 데이터형과 코드에서 식별해서 사용하기 위한 용도인 변수명을 사용하여 표현한다.

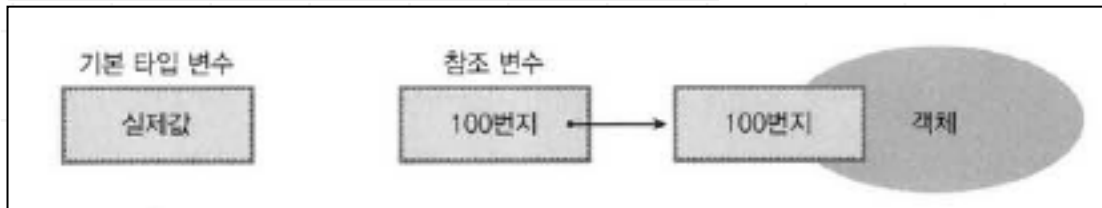
변수 선언은 사용자가 지정한 식별자로서 식별자 규칙에 맞게 지정해야 된다.
주의할 점은 동일한 변수이름으로 중복 선언은 불가능하다. (식별 불가)

문법:

데이터형 변수명;

```
예> int num;        // 기본형 변수  
    String name;    // 참조형 변수  
  
    int age=10,height,weight; //권장 안함.
```

기본형 변수와 참조형 변수 차이점



4. 변수 (variable)

JDK 21

2) 변수 초기화 (initialization)

선언된 변수에 처음으로 값을 입력시키는 작업을 의미한다.

문법:

변수명=값;

```
예> int num;  
    String name;  
  
    num=10;  
    name="홍길동";
```

다음과 같이 변수선언과 초기화 작업을 한꺼번에 처리 할 수 있다.

```
예> int num = 10;  
    String name = "홍길동";
```

3) 변수값 변경

변수에 저장된 데이터는 항상 동일한 데이터를 유지하지 않고 프로그램이 실행되면서 언제든지 변경될 수 있다.
결국 저장된 값이 변경될 수 있기 때문에 변수 라고 부른다.

예>

```
int age = 10;
```

```
..
```

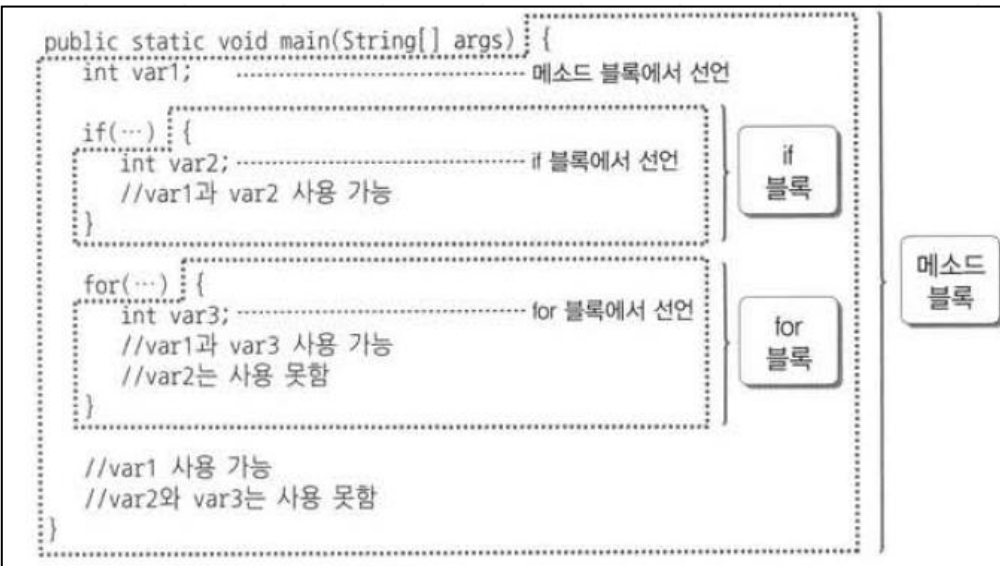
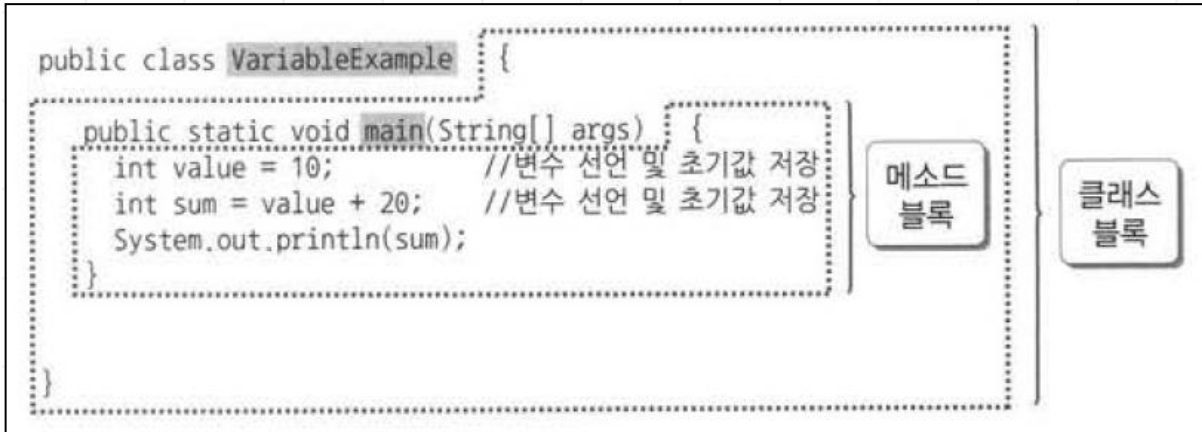
```
age = 20; // 프로그램 실행 중에 값 변경됨.
```

4. 변수 (variable)

JDK 21

변수의 사용 범위 (scope)

변수는 블록({})내에서 선언되고 사용된다. (블록 scope)



변수 종류

1. 로컬변수(local variable)
2. 인스턴스 변수(instance variable)
3. 클래스 변수(static variable)

값이 변경될 수 있는 것을 변수라고 하며
값을 변경하지 못하는 것을 상수라고 한다.
final 키워드를 사용하여 변수를 상수로 만든다.
일반적으로 변수명과 구분하기 위하여 상수명은 대문자로 설정한다.

문법:

```
final 데이터형 상수명=값;
```

예>

```
public static final int NUM = 30;
```

기본 타입	최대값 상수	최소값 상수
byte	Byte.MAX_VALUE	Byte.MIN_VALUE
short	Short.MAX_VALUE	Short.MIN_VALUE
int	Integer.MAX_VALUE	Integer.MIN_VALUE
long	Long.MAX_VALUE	Long.MIN_VALUE
float	Float.MAX_VALUE	Float.MIN_VALUE
double	Double.MAX_VALUE	Double.MIN_VALUE

서로 다른 종류의 데이터가 프로그램 실행 중에 원래의 데이터형을 버리고 새로운 데이터 형으로 변경되는 것을 의미한다.
단, 같은 종류의 데이터끼리만 변경이 가능하다.

데이터 형 변환 2가지 형태는 다음과 같다.

1) 묵시적 형변환

- 자동형 변환 또는 promotion이라고 한다.
- 기본 데이터형 또는 참조 데이터형(상속관계) 모두 가능하다.
- 기본적으로 수치형은 데이터형 변환이 데이터형이 큰 쪽으로 자동으로 변환이 가능하다. (up casting)
- char 문자형은 int 정수형으로 변환이 가능하다.

byte -> short -> int -> long -> float -> double
char -> int

-정수형 중에서 int 보다 작은 타입의 연산결과는 기본값이 int로 반환된다.

예>

```
short s  = 1;  
short s2 = 2  
int result = s + s2;
```

-작은 데이터형과 큰 데이터형의 연산결과는 큰 데이터형으로 반환된다.

예>

```
double result = 10/3.0;
```

2) 명시적 형 변환

- 강제형 변환 또는 type casting이라고 한다.
- 작은 데이터 형으로 변환되기 때문에 down casting이라고도 부른다.
이 방법은 데이터 손실이 발생 될 수도 있다.

다음과 같은 형 변환 연산자를 사용한다.

문법:

(데이터형)값;

```
예> int num = 10;  
    short s = (short)num;
```

출력 대상은 다양하다.
모니터에 출력하는 경우를 표준출력(standard output) 이라고 한다.
반대 개념인 표준입력(standard input)은 키보드로서 입력 받는 것을 의미한다.

문법:

```
System.out.println(값);
```

```
System.out.print(값);
```

```
System.out.printf(" 형식문자 %s %d", 값1, 값2);
```

3장. 연산자

- 1) 연산자
- 2) 산술 연산자
- 3) 대입 연산자
- 4) 비교 연산자
- 5) 논리 연산자
- 6) 증감 연산자
- 7) 3항 연산자

연산자(operator)란 자료의 가공을 위해 정해진 방식에 따라 계산하고 결과를 얻기 위한 행위를 의미하는 기호들의 총칭이다.

연산되는 데이터는 피연산자(operand)라고 한다.

종류	연산자	우선순위
증감 연산자	++, --	1순위
산술 연산자	+, -, *, /, %	2순위
시프트 연산자	>>, <<, >>>	3순위
비교 연산자	>, <, >=, <=, ==, !=	4순위
비트 연산자	&, , ^, ~	~만 1순위, 나머지는 5순위
논리 연산자	&&, , !	!만 1순위, 나머지는 6순위
조건(삼항) 연산자	?, :	7순위
대입 연산자	=, *=, /=, %=, +=, -=	8순위

- 4칙 연산(+, -, *, /)과 나머지 값을 구하는 연산자(%)가 있다.

구분	연산자	의미
산술 연산자	+	더하기
	-	빼기
	*	곱하기
	/	나누기
	%	나머지 값 구하기

정수값으로 나누기하면 결과값은 정수값으로 반환. (예 > $10/3=>3$)
실수값으로 나누어야 실수값으로 반환 (예 > $10/3.0=3.3333333$)

- 특정한 상수 값이나 변수 값 또는 객체를 변수에 저장 시킬 때 사용하는 연산자이다.

구분	연산자	의미
대입 연산자	=	연산자를 중심으로 오른쪽 변수값을 왼쪽 변수에 대입한다.
	+=	왼쪽 변수에 더하면서 대입한다.
	-=	왼쪽 변수값에서 빼면서 대입한다.
	*=	왼쪽 변수에 곱하면서 대입한다.
	/=	왼쪽 변수에 나누면서 대입한다.
	%=	왼쪽 변수에 나머지 값을 구하면서 대입한다.

```

a=b;      // a에 b를 저장(할당,대입)함.
a+=b;     // a=a+b; 동일
a-=b;     // a=a-b; 동일
a*=b;     // a=a*b; 동일
a/=b;     // a=a/b; 동일
a%=b;     // a=a%b; 동일
    
```

- 변수나 상수의 값을 비교할 때 쓰이는 연산자로서 결과가 항상 true 또는 false인 논리값(boolean)으로 반환된다.

구분	연산자	의미
비교 연산자	>	크다.
	<	작다.
	>=	크거나 같다.
	<=	작거나 같다.
	==	피연산자들의 값이 같다.
	!=	피연산자들의 값이 같지 않다.

- true나 false인 논리 값을 가지고 연산하는 연산자이다.
- 하나 이상의 처리 조건이 있어야 하며 먼저 처리되는 조건에 따라 다음의 처리 조건을 처리할지 안 할지를 결정하는 말 그대로 논리적인 연산자이다.

구분	연산자	의미	설명
논리 연산자	&&	and(논리곱)	주어진 조건들이 모두 true일 때만 true를 나타낸다.
		or(논리합)	주어진 조건들 중 하나라도 true이면 true를 나타낸다.
	!	not(부정)	true는 false로 false는 true로 나타낸다.

○ Short-circuit logical 연산자

&& 또는 || 연산자 앞의 논리값만 확인해서 최종적인 결과가 반환되는 메커니즘.

연산자	설명
&&	선조건이 true일 때만 후조건을 실행하며 선조건이 false이면 후조건을 실행하지 않는다.
	선조건이 true이면 후조건을 실행하지 않으며 선조건이 false일 때만 후조건을 실행한다.

- 1씩 증가 또는 감소시키는 연산자이다.
- 주의할 점은 다른 연산자와 같이 사용시 증감연산자가 변수 앞에 위치 하느냐? 아니면 변수 뒤에 위치 하느냐? 에 따라서 결과값이 달라 질 수 있다. (전치 및 후치)

구분	연산자	의미
증감 연산자	++	1씩 증가시킨다.
	--	1씩 감소시킨다.

- 조건을 이용해서 임의의 값을 얻을 때 사용.
- 중첩도 가능하다.

구분	연산자	의미	구성
조건 연산자	? :	제어문의 단일 비교문과 유사하다.	조건식 ? 참값 : 거짓값

4장. 제어문

1. 제어문의 종류

2-3. (비)실행문

4-6. if문

7. switch문

8. for 반복문

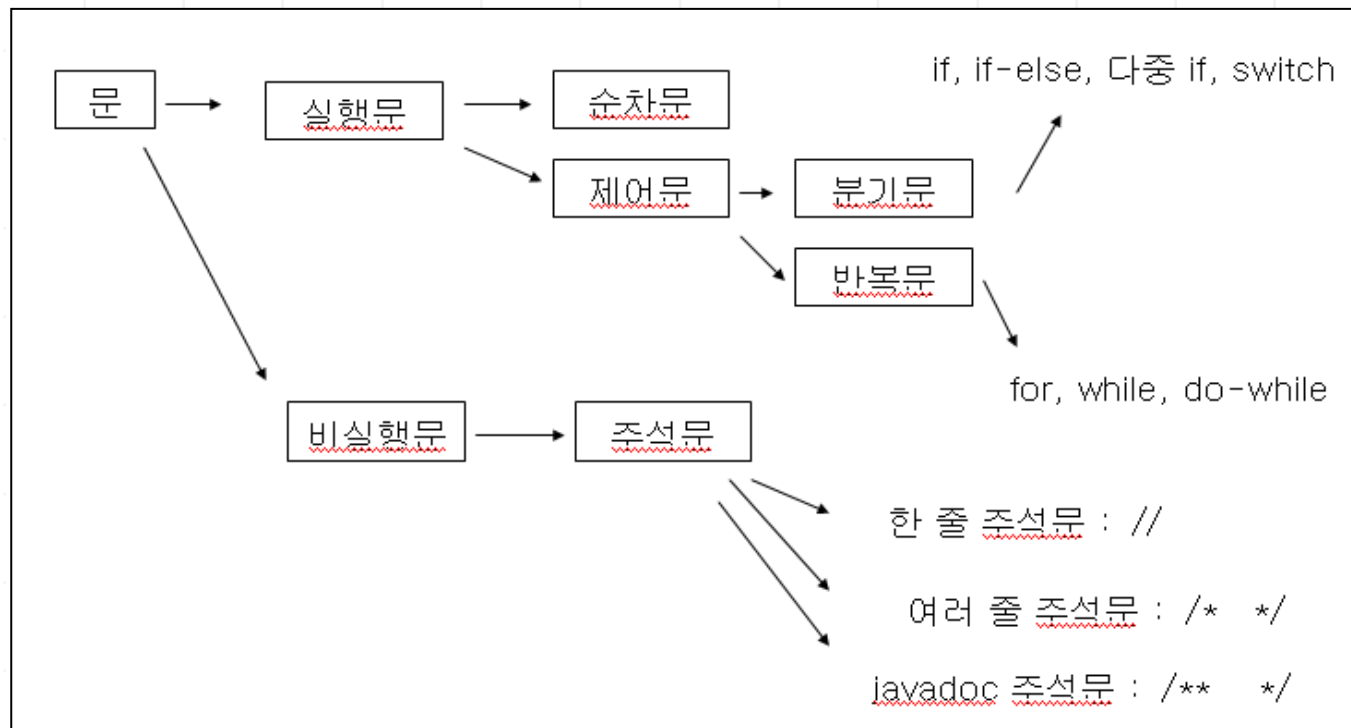
9. while 반복문

10. do~while 반복문

11. 중첩 반복문

12. break, continue

문(statement)은 자바 프로그램을 개발하기 위해서 자바소스코드에 입력시키는 문장을 의미한다.



○ 주석문

- 실행이 안되기 때문에 실제 프로그램에 영향을 주지 않으며 단지 소스코드의 기능이나 동작을 설명하기 위해 사용되는 문장이다.

주석 종류	의미	설명
// 주석문	단행 주석처리	현재 행에서 //의 뒷문장부터 주석으로 처리된다.
/* 주석문 */	다행 주석처리	/*에서 */ 사이의 문장이 주석으로 처리된다.
/** 주석문 */	HTML 문서화 주석처리	/**에서 */ 사이의 문장이 주석으로 처리된다. 장점은 HTML 문서화로 주석이 처리되므로 API와 같은 도움말 페이지를 만들 수 있다.

○ 순차문

- 메서드내의 문장 중에서 순차적으로 실행되는 문장을 의미한다.
- 반드시 ;(세미콜론)으로 끝나며 자바 소스코드의 대부분이 순차문에 해당된다.

○ 제어문

- 프로그램의 흐름에 영향을 주고 때에 따라서 제어가 가능하도록 하는 것이 바로 제어문이다.
- 모든 제어문은 중첩이 가능하다.

분기문 (비교문)

주어진 조건의 결과에 따라 실행 문장을 다르게 하여 전혀 다른 결과를 얻기 위해 사용되는 제어문이다.

종류로는 단일 if문, if~else문, 다중 if문, switch문이 있다.

반복문

특정한 문장을 정해진 규칙에 따라 반복처리하기 위한 제어문이다.

종류로는 for문, while문, do~while문이 있다.

break문

반복문 안에서 쓰이며 반복문을 빠져나갈 때 쓰이는 제어문이다.

switch문에서 사용시 switch 문을 빠져나간다.

continue문

반복문 안에서 쓰이며 현재 진행되는 반복 회차를 포기하고 다음 회차로 이동한다.

(반복문 안의 블록 끝으로 이동)

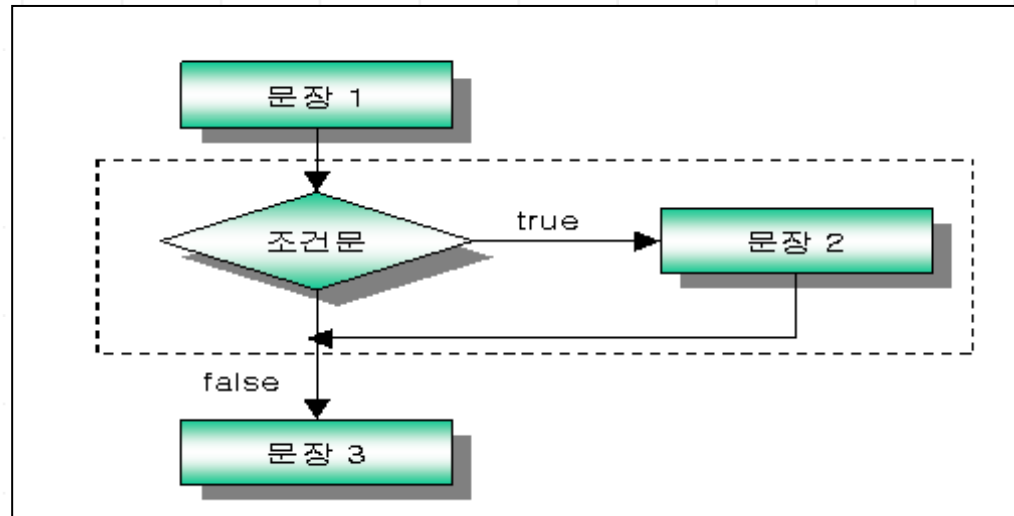
5. 단일 if문

JDK 21

주어진 조건을 만족하는 경우에만 특정 문장을 수행하도록 제어하는 문이다.

문법

```
문장1;  
if(조건식){  
    문장2;  
}  
문장3;
```



➔ 문장2는 조건식의 결과에 따라서 실행 여부가 결정된다.

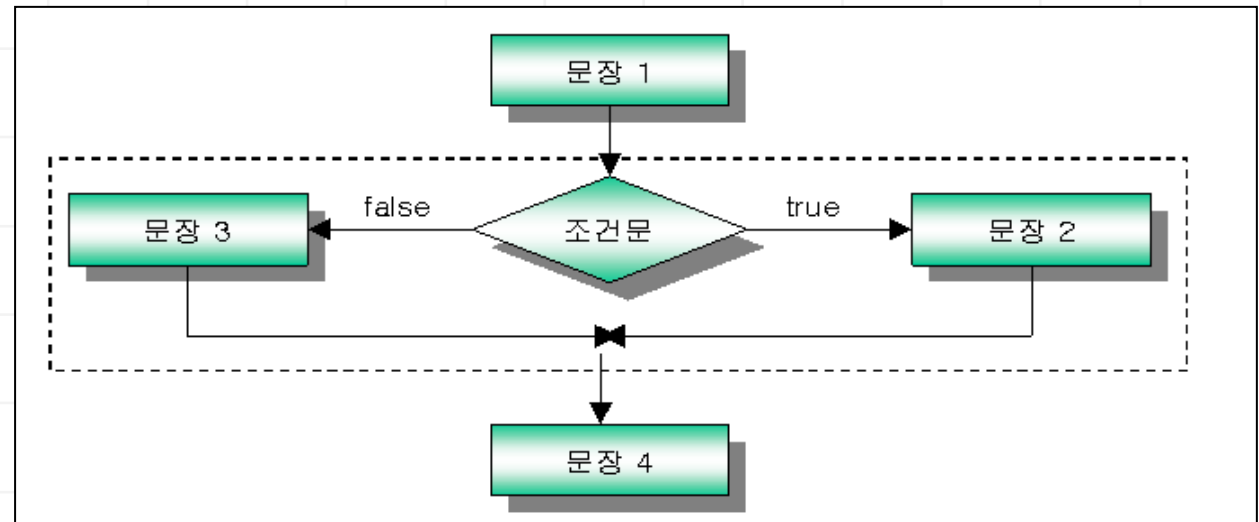
```
if(조건식)  
    do something..    // 수행 문장이 하나일 경우 { } 생략 가능
```

6. if ~ else 문

조건식의 결과에 따라서 실행되는 문장이 서로 다른 경우에 사용한다.

문법

```
문장1;  
if(조건식){  
    문장2;  
}else{  
    문장3;  
}  
문장4;
```

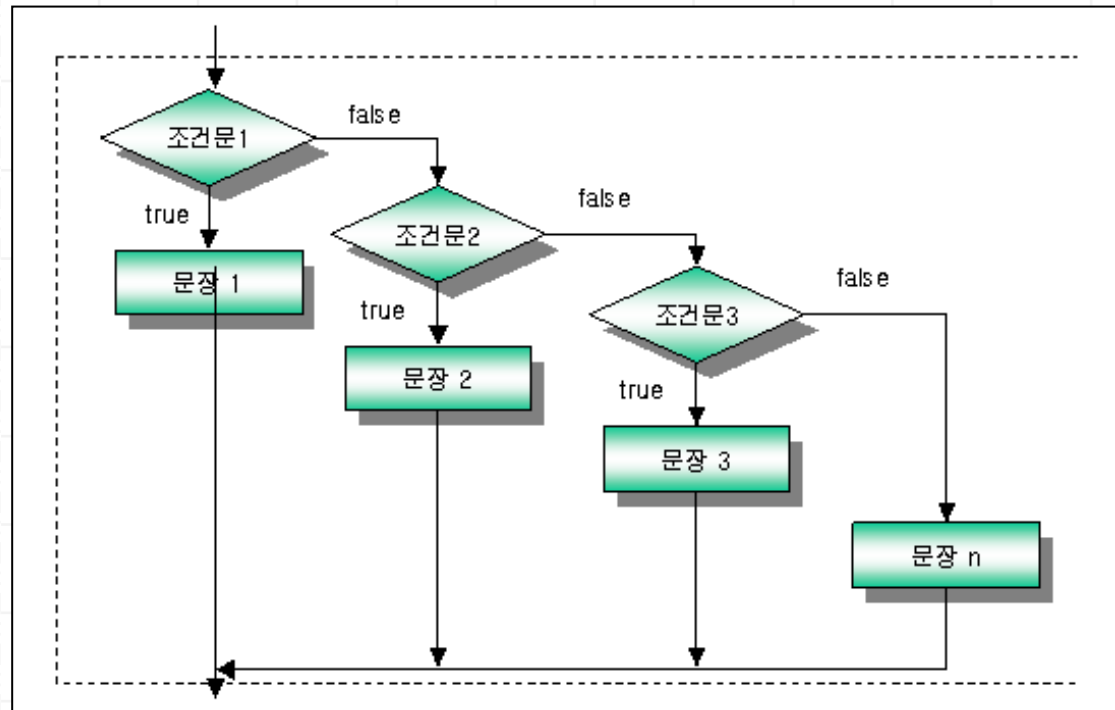


동작은 조건식이 참이면 문장2가 실행되고 거짓이면 문장3이 실행된다.

비교할 조건식이 여러 개인 경우에 사용된다.

문법

```
if(조건식1){  
    문장1;  
}else if(조건식2){  
    문장2;  
}else if(조건식3){  
    문장3;  
}else{  
    문장n;  
}
```



문장3이 실행되기 위해서는 조건식1과 조건식2는 false 이고 조건식3은 true 이다.

다중 if ~ else 문과 비슷한 용도로 사용된다. (동등비교)

문법

```
switch(인자값) {  
    case 조건값1 :  
        실행문; break;  
    case 조건값2 :  
        실행문; break;  
    case 조건값3 :  
        실행문; break;  
    default :  
        실행문;  
}
```

인자값 위치에서 사용 가능한 데이터 형은 6가지이다.

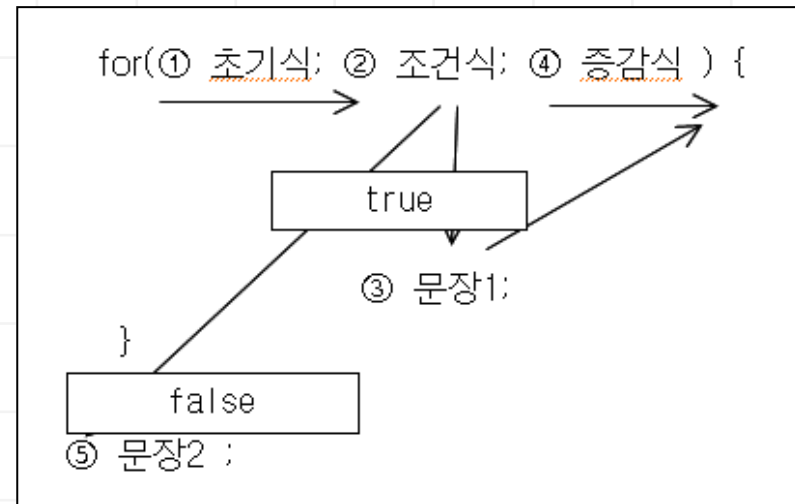
- byte , short , int , char , String , enum

case문의 값은 각각 달라야 하며 값의 크기와 순서는 무관하다.

지정된 횟수만큼 반복 처리하는 제어문이다.
초기식, 조건식, 증감식으로 구성된다.
일반적으로 반복 횟수가 예측 가능할 때 주로 사용된다.

문법

```
for(초기식;조건식;증감식){  
    문장1;  
    ...  
}  
  
문장2;
```



for문과 문법적인 형태만 다르며 동일한 방식으로 동작한다.

차이점은 for문은 초기식,조건식,증감식의 지정된 위치가 존재하지만 while문은 조건식만 정해져 있기 때문에 초기식과 증감식은 적당한 위치에 명시적으로 지정해야 된다. 예측 불가능한 형태의 반복문에 주로 사용된다.

문법

```
    초기식;
    ..
while(조건식){
    문장;
    ..
    증감식;
}
```

while문과 비슷하지만 차이점은 조건식이 일치하지 않더라도 반드시 한번은 문장이 실행된다.
조건이 일치하지 않더라도 적어도 한번은 꼭 수행되어야 하는 경우에 사용 가능하다.

문법

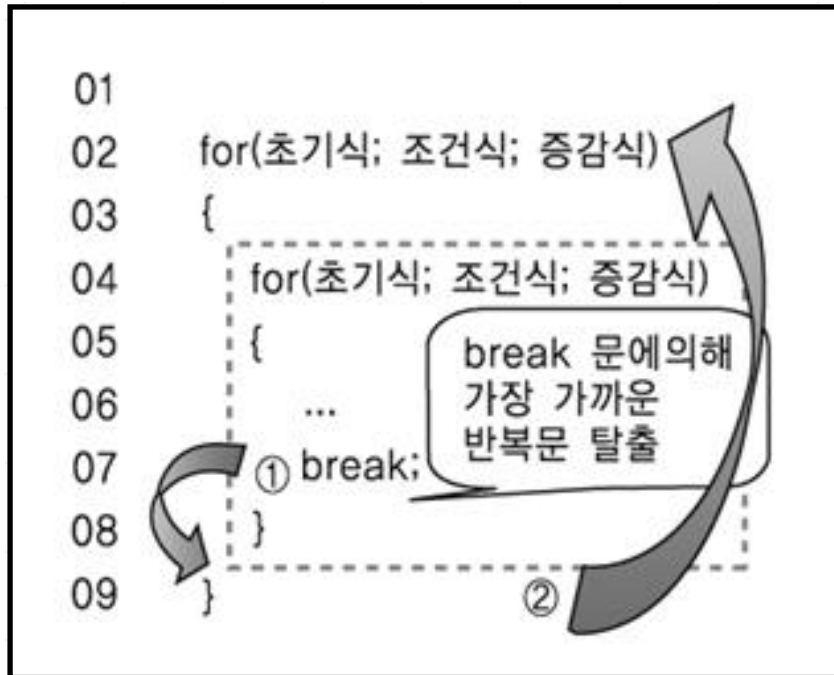
```
    초기식;  
    ..  
do{  
    문장;  
    ..  
    증감식;  
}while(조건식);
```

반복문내에 또다른 반복문을 중첩으로 지정하여 사용하는 문이다.
for문, while문, do~ while문 모두 중첩이 가능하다.

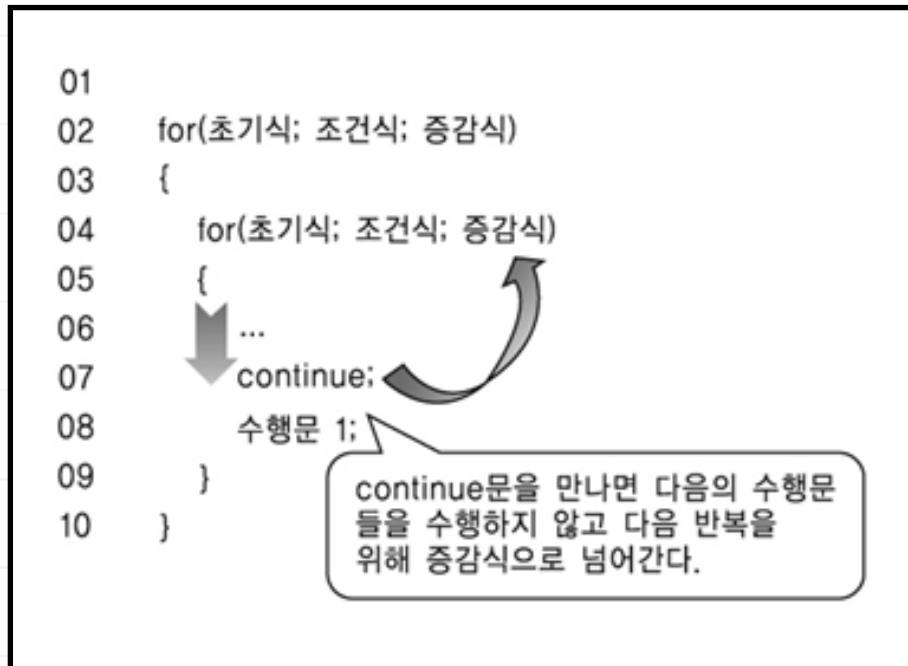
문법

```
for(초기식1 ; 조건식1 ; 증감식1) {  
    for(초기식2 ; 조건식2 ; 증감식2){  
        명령어2;  
    }  
    명령어1;  
}  
명령어3;
```

- 가장 가까운 반복문을 빠져나올 때 사용되는 제어문이다.



- 반복문을 중단하기 위해 사용되는 것이 아니라 continue문 이하의 수행문들을 포기하고 다음 회차의 반복을 수행하기 위한 제어문이다.



5장. 배열

1. 배열
2. 1차원 배열 사용법
3. 2차원 배열 사용법

배열 목적

같은 타입의 데이터가 여러 개인 경우에는 데이터 개수만큼 변수가 필요하다.
여러 개의 변수를 사용하면 관리가 어려워진다.
하지만 배열은 단 하나의 변수로 많은 데이터를 효율적으로 관리할 수 있다.

변수 1개



변수 3개



배열 1개



- 같은 종류의 데이터만 저장이 가능하다.
- 기본 데이터형 및 참조 데이터형 모두 배열로 관리 가능하다.
- 배열은 참조 데이터이기 때문에 반드시 `new` 키워드를 사용하여 생성하고 배열 요소들은 자동으로 초기화 된다.
- 생성된 배열의 각 요소는 `배열명[index]` 로 접근할 수 있다.
`index`는 첨자로서 0 부터 시작한다.
- 배열의 길이는 `배열명.length` 로 구한다.
배열의 크기를 넘어서는 요소 접근시 `ArrayIndexOutOfBoundsException` 예외가 발생된다.
- 한번 생성된 배열의 크기 변경은 불가능하다.

사용방법 1:

가. 배열 선언

문법:

```
데이터형 [] 배열명; | 데이터형 배열명 [];
```

```
예> int [] num;      String [] name;
```

나. 배열 생성

문법:

```
배열명 = new 데이터형[크기];
```

```
예> num = new int[3];      name = new String[2];
```

다음과 같이 선언과 생성을 한꺼번에 처리할 수도 있다.

```
int [] num = new int[3];
```

다. 배열 초기화

문법:

배열명[index] = 값;

```
예 > num[0] = 10;      name[0] = "홍길동";  
      num[1] = 20;      name[1] = "이순신";  
      num[2] = 30;
```

배열 길이: 배열명.length

```
System.out.println( num.length );    // 3  
System.out.println( name.length );    // 2
```

사용방법 2:

리터럴(literal)를 이용하여 생성하는 방법이다.

배열 선언, 생성, 초기화를 한꺼번에 처리할 수 있도록 코드를 작성하며 주의할 점은 초기화를 나중에 할 수 없다.

문법:

```
데이터형 [ ] 배열명 = { 값 , 값2 , 값3 };
```

예>

```
int [] num = { 10 , 20, 30 };
```

```
String [] name = { "홍길동", "이순신" , "이순신" };
```

사용방법 3:

사용방법1 과 사용방법2 를 혼합한 방법이다.
주의할 점은 명시적으로 배열 크기를 지정하면 안된다.

문법:

```
데이터형 [ ] 배열명 ;  
배열명 = new 배열명[ ] { 값, 값2 , 값3 };
```

예>

```
int [] num = new int[] { 10, 20 ,30 };
```

```
String [] name = new String[] { “홍길동”, “이순신” , “이순신” };
```

사용방법 1:

가. 배열 선언

문법:

```
데이터형 [][] 배열명; | 데이터형 배열명 [][];
```

```
예> int [][] num;      String [][] name;
```

나. 배열 생성 (정방향)

문법:

```
배열명 = new 데이터형[행크기][열크기];
```

```
예> num = new int[3][2];      name = new String[2][2];
```

다음과 같이 선언과 생성을 한꺼번에 처리할 수도 있다.

```
int [][] num = new int[3][2];
```

다. 배열 초기화

문법:

```
배열명[행index][열index] = 값;
```

```
예> num[0][0] = 10;      name[0][0] = "홍길동";
    num[0][1] = 20;      name[0][1] = "이순신";
    num[1][0] = 30;      name[1][0] = "유관순";
    num[1][1] = 40;      name[1][1] = "강감찬";
    num[2][0] = 50;
    num[2][1] = 60;
```

행의 길이: 배열명.length

1행의 열의 길이: 배열명[0].length

2행 1열의 길이: 배열명[1][0].length

사용방법 2:

가. 배열 선언

문법:

```
데이터형 [][] 배열명; | 데이터형 배열명 [][];
```

```
예> int [][] num;      String [][] name;
```

나. 행 배열 생성 (비정방형, **행 크기만 지정**하고 열 크기는 나중에 동적으로 지정)

문법:

```
배열명 = new 데이터형[행크기][];
```

```
예> num = new int[3][];      name = new String[2][];
```

다. 열 배열 생성

문법:

```
배열명[행index] = new 데이터형[열크기];
```

```
예>   num[0] = new int[2];   name[0] = new String[1];  
      num[1] = new int[3];   name[1] = new String[2];
```

라. 배열 초기화

문법:

```
배열명[행index][열index] = 값;
```

```
예>   num[0][0] = 10;          name[0][0] = "홍길동";  
      num[0][1] = 20;          name[1][0] = "이순신";  
      num[1][0] = 30;          name[1][1] = "유관순";  
      num[1][1] = 40;  
      num[1][2] = 50;
```

사용방법 3:

배열 선언, 생성, 초기화를 한꺼번에 작성한다.

주의할 점은 초기화를 나중에 할 수 없다.

문법:

```
데이터형 [][] 배열명 = { { 값, 값2 }, { 값3 }, { 값4, 값5 } };
```

예>

```
int [][] num = { { 10 , 20 } , { 30 } , { 40 , 50 } };
```

```
String [][] name = { { "홍길동", "이순신" } , { "이순신" } };
```

사용방법 4:

사용방법1 과 사용방법2,3 를 혼합한 방법이다.
주의할 점은 명시적으로 배열 크기를 지정하면 안된다.

문법:

```
데이터형 [][] 배열명  
= new 데이터형[][]{ { 값, 값2 }, { 값3 }, { 값4, 값5 } };
```

예>

```
int [][] num = new int[][]{ {10 , 20 } , {30 } , { 40 , 50 } };
```

```
String [][] name  
= new String[][]{ {"홍길동", "이순신"} , {"이순신"} };
```

6장. 객체 및 클래스

- 1. 객체지향 프로그래밍
- 2-3. 객체와 클래스
- 4-5. 클래스 구성요소, 객체생성
- 6-7. 메서드, 오버로딩 생성자 및 메서드
- 8. 메서드 호출시 파라미터 전달 방법
- 9-10. this, package와 import 문
- 11-12. static 키워드

1) 객체(object)란?

주체(subject)가 바라보는 현실세계의 모든 사물 및 대상을 의미한다.

2) 객체(object) 구성 요소

속성 및 동작으로 구성된다.

예> 고양이 객체

속성: 이름, 나이, 체중, ..

동작(기능): 먹기, 뛰기, ..

3) 객체지향 프로그래밍 (OOP, Object Oriented Programming)

주변의 많은 것들을 객체화 해서 프로그래밍 하는 방법론.

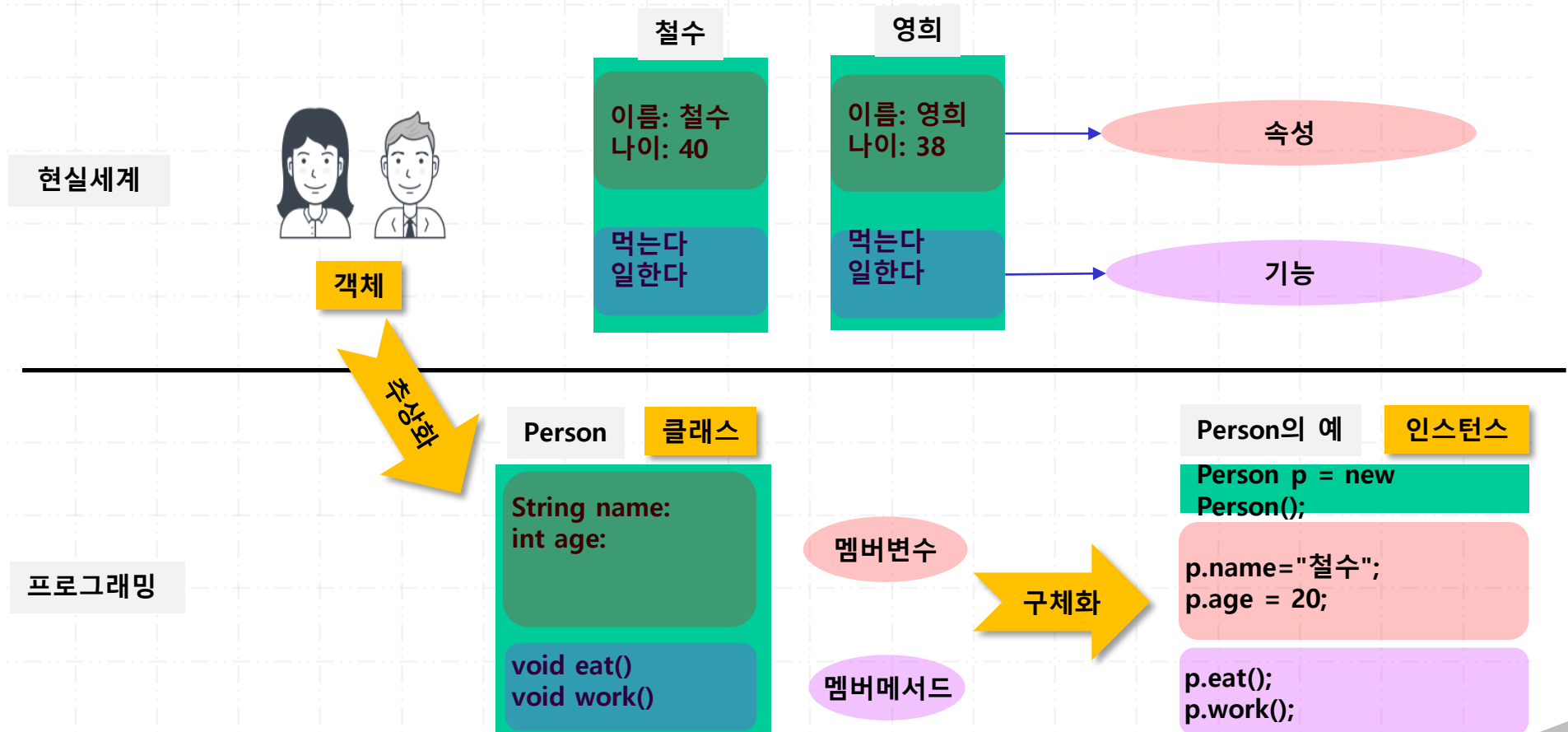
고양이 객체	----->	Cat 클래스
속성	----->	변수
동작(기능)	----->	메서드

2. 객체와 클래스

JDK 21

현실 세계의 객체, 클래스, 인스턴스(instance)의 관계

- 현실의 객체가 갖는 속성과 기능은 추상화(abstraction) 되어 클래스에 정의.
- 클래스는 구체화 되어 인스턴스(instance)가 된다.



문법

```
public class 클래스명 {  
    // 인스턴스 변수  
    // 메서드  
    // 생성자  
}
```

클래스의 구성요소 3가지

- 인스턴스 변수 (instance variable) : 멤버변수
용도: 클래스의 특성을 표현하기 위한 속성값을 저장할 때 사용.
- 메서드 (method) : 멤버 메서드
용도: 주로 인스턴스 변수의 값을 관리하는 역할.(저장 및 수정 , 조회)
- 생성자 (constructor)
용도: 인스턴스 변수에 값을 가장 처음 저장하는 역할. (초기화 역할)

4. 지정자 (modifier)

JDK 21

특정 목적을 위해서 사용하는 키워드로서 클래스, 변수, 메서드에 사용할 수 있고 생략도 가능하다.

가. 일반 지정자 (modifier)

- static
- final
- abstract

나. 접근 지정자 (access modifier)

- public
- protected
- (default)
- private

- 객체를 잘 표현할 수 있는 의미 있는 명사형으로 지정한다.
- 반드시 첫 글자는 대문자로 지정한다.
소문자로 지정해도 문법적으로는 문제가 없지만 관습적으로 첫 글자는 대문자로 지정해서 사용한다.
- 파일 저장시에는 반드시 '클래스명.java' 로 저장해야 된다.

예>

학생객체: Student.java

사원객체: Employee.java

6. 인스턴스 변수 (instance variable)

JDK 21

문법:

```
public class 클래스명{  
    [접근지정자] 데이터형 변수명;  
    ..  
}
```

- 객체의 속성 데이터를 저장하는 용도이다.
- 클래스를 사용하기 위해서는 반드시 객체생성(인스턴스화)을 해야 하는데 이때 생성되는 변수이다. (객체 생성시 매번 생성된다.)
- 일반 변수명 규칙을 따른다. (의미 있는 명사형, 소문자, 중복선언 불가)
- heap 메모리에 저장된다.

가. 로컬 변수

- 메서드 안에서 선언된 변수이다.
- 메서드가 호출될 때 생성되고 메서드가 종료될 때 제거된다.
stack 메모리에 저장된다. 임시적으로 사용된다.
- 접근 지정자를 사용하지 못한다.
- 기본적으로 메서드 블록({}) scope를 따른다.
- 반드시 사용 전에 초기화해야 된다.

나. 인스턴스 변수

- 메서드 밖에서 선언된 변수이다.
- 객체 생성할 때 생성되고 객체가 제거될 때 삭제된다.
- 객체 생성후에 참조변수명.인스턴스변수 형식으로 접근한다.
- heap 메모리에 저장된다. 클래스의 인스턴스 정보(속성)를 저장하는 용도이다.
- 사용 전에 초기화 하지 않으면 기본값으로 자동 설정된다.
- 기본적으로 클래스 블록({ }) scope를 따른다.

다. 클래스 변수

- 메서드 밖에서 선언된 변수로서 **static** 키워드를 사용한다.
- 프로그램이 실행될 때 자동으로 생성되고, 프로그램이 종료 될 때 삭제된다.
- 프로그램은 한번만 실행되기 때문에 클래스 변수도 단 한번만 생성된다.
객체 생성 전에 생성이 된다. 따라서 인스턴스와 무관한다.
- 클래스명.변수 형식으로 접근한다.
- 일반적으로 데이터 누적용으로 사용된다.
- method Area 영역에 저장된다. (**static** 키워드 및 상수 저장 영역)
- 초기화 하지 않으면 자동으로 기본값으로 설정된다.

예>

```
public class 클래스명 {  
    int num;           // 인스턴스 변수  
    static int age;     // 클래스 변수  
    ..  
}
```

문법

```
public class 클래스명{  
    ...  
    [접근 지정자] 클래스명([인자,인자2]){  
        ...  
    }  
}
```

- 인스턴스 변수에 데이터를 초기화하는 역할이다.
- 메서드와 매우 비슷한 특징을 갖는다. 가장 큰 차이점은 리턴 타입이 없으며 생성자명은 반드시 클래스명으로 지정해야 된다.
- 자동으로 기본 생성자(default constructor)가 생성된다.
만약 개발자가 명시적으로 생성자를 작성하면 기본 생성자는 더 이상 자동으로 제공되지 않는다.
기본생성자 형식: `public 클래스명(){}`
- 필요에 의해서 생성자를 여러 개 작성할 수도 있다. (오버로딩 생성자)

문법

```
클래스명 참조변수명 = new 클래스명( [인자,인자2] );
```

- 정의한 클래스를 이용하여 실객체를 메모리에 올리는 방법이다.
- 메모리에 올라간 클래스의 구현체(실 객체)를 인스턴스(instance)라고 한다.
- heap 메모리에 클래스의 멤버(인스턴스 변수/ 메서드)가 생성된다. 인스턴스 변수는 new할 때마다 매번 생성되며 메서드는 공유해서 사용된다.
- 참조변수에는 heap 메모리에 저장된 클래스의 인스턴스 위치값이 저장되어 있기 때문에 이 변수를 이용하여 클래스의 멤버를 접근할 수 있다.
예> 참조변수.멤버
- 같은 클래스 안의 변수와 메서드는 .(dot)없이 바로 접근이 가능하다.

개념

객체생성후에 heap 메모리에 생성된 자신의 인스턴스를 가리킨다.
다른 클래스에서는 참조변수명을 사용하여 생성된 인스턴스를 참조할 수 있지만
자신이 참조할 때는 this 키워드를 사용한다.
일반적으로 생략해서 사용된다.

명시적 사용하는 경우1: 인스턴스 변수명과 로컬변수명이 동일한 경우

예 >

```
String name;  
public Student( String name, int age , String addr){  
    this.name = name;  
    this.age = age;  
    this.address = addr;  
}
```

명시적 사용하는 경우2: 오버로딩 생성자에서 다른 생성자 호출하는 경우

예 >

```
public Student( String name, int age , String addr){
    this.name = name;
    this.age = age;
    this.address = addr;
}
public Student( String name, int age ){
    this( name, age, "서울");
}
public Student( String name ){
    this( name , 20 );
}
```

주의할 점은 반드시 생성자 첫 라인에서 호출해야 된다.
자연스럽게 중복된 초기화 코드가 제거된다.

문법

```
public class 클래스명{  
    [지정자] 리턴타입 메서드명([변수,변수2]){  
        ...  
        [return [값];]  
    }  
}
```

- 일반적으로 인스턴스 변수에 저장된 데이터를 수정 및 조회하는 역할이다.
- 메서드는 반드시 객체 생성 후에 명시적으로 호출해서 사용된다.
- 메서드 호출 시 주의할 점은 반드시 메서드명과 인자리스트(순서,개수,타입)가 동일해야 된다.
- 호출해서 수행된 메서드는 모든 작업이 끝나면 호출한 곳으로 돌아간다.
- 메서드 동작을 중간에서 멈추기 위하여 리턴값 없이 return; 문만 사용할 수 있다.

- 메서드 역할에 따른 분류

가. getter 메서드

- 특정 결과값을 얻고자 작성한 메서드를 의미한다.
- 메서드명은 get변수명(첫 글자는 대문자)형식으로 지정하는 것을 권장.

나. setter 메서드

- 특정값을 설정(넘겨 줄 때)하고자 할 때 사용된다.
- 메서드명은 set변수명(첫글자는 대문자)형식으로 지정하는 것을 권장.

다. 추가 기능 메서드

- getter 및 setter 메서드가 아닌 임의의 기능이 필요할 때 추가로 지정 가능하다.

메서드와 생성자는 이름과 인자로 식별이 가능하기 때문에 동일한 이름의 메서드와 생성자를 여러 개 지정될 수 있다.
이것을 오버로딩 메서드/오버로딩 생성자라고 한다.

구별하기 위한 규칙

이름은 동일하지만 반드시 인자리스트(순서, 타입, 개수)가 달라야 된다.

예 >

```
public Student(){}  
public Student(String n){}  
public Student(String n, int a){}  
public Student( int a, String n){}  
  
public void method(){  
public void method(int n){}
```

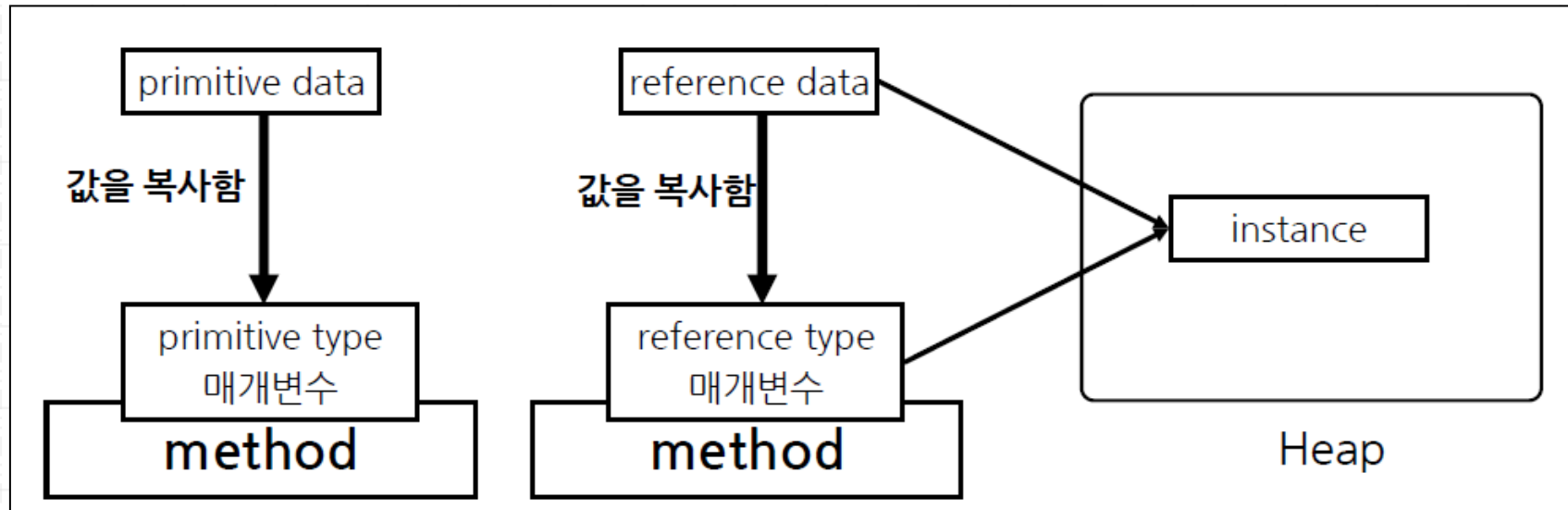
13. 메서드 호출시 파라미터 전달 방법

JDK 21

Call by Value / Call by Value Reference

자바의 매개변수는 값 복사를 통해서만 전달된다.

메서드 호출시 매개변수로 전달되는 값은 기본 데이터인 경우에는 변수에 저장된 실제값이 전달되고 참조 데이터형인 경우에는 변수에 저장된 객체의 주소값이 전달된다.



윈도우의 파일을 효율적으로 관리하기 위해서 폴더를 사용하는 것처럼 클래스 파일들을 효율적으로 관리하기 위해서 package을 사용할 수 있다.

문법

```
package 패키지명.패키지명2;
```

- 클래스 선언보다 먼저 작성하고 한번만 선언할 수 있다.
- 패키지명은 계층구조를 가질 수 있다.
- 중복되면 안되기 때문에 일반적으로 도메인명으로 지정한다.
- 패키지가 다르면 접근 불가능하다. (import 문으로 해결)
- API 패키지명으로 사용자 패키지명을 사용하지 않는다.
- 시스템이 제공하는 API는 모두 package 형태로 제공된다.
따라서 사용하기 위해서는 반드시 import 해야 된다.
API 중에서 유일하게 java.lang 패키지는 자동으로 import 됨.

기본적으로 package가 서로 다르면 접근이 불가능하다.
이것을 가능하게 하는 방법이 import 문을 사용하는 것이다.

문법

```
import 패키지명.패키지명2.클래스명;  
import 패키지명.패키지명2.*;           // 권장안함
```

- 클래스에서 여러 번 사용이 가능하다.
- 클래스 선언보다 먼저 사용하고 package문 보다는 나중에 사용한다.
- 클래스명 대신에 * 를 사용할 수 있지만 가독성 때문에 권장하지 않는다.
- 시스템이 제공하는 API중에서 java.lang 패키지는 유일하게 자동으로 import 된다.
따라서 java.lang 패키지를 제외하고는 모두 import 해서 사용해야 된다.

개요

- 클래스, 변수, 메서드 지정자(modifier)로 사용 가능 하다.
- 프로그램 실행할 때 단 한번 생성되고 프로그램이 종료될 때 제거된다.
- 객체생성과 무관하기 때문에 객체 생성 없이도 사용이 가능하다. (클래스명으로 접근)
- static 메서드는 Overriding이 불가능하고 인스턴스 변수를 접근하지 못한다.
- static 블록 이용한 초기화 작업이 가능하다.

사용방법

- 클래스 : inner 클래스에서만 사용 가능.
- 메서드 : 객체 생성 없이 메서드 사용하기 위함.
예> `int num = Integer.parseInt("123");`
- 변수: 데이터 누적용.
프로그램 내에서 특정 데이터 값을 계속 유지할 목적이라면 static변수가 적합하다.

17. static import 및 static 블록

JDK 21

static으로 정의된 변수와 메서드를 매번 클래스 이름으로 접근하지 않고도 사용할 수 있는 방법이다.

예>

```
import static java.lang.Math.PI;
import static java.lang.Integer.parseInt;

System.out.println( Math.PI );
System.out.println( PI );           // 가독성은 떨어짐.

String s = parseInt( "123");
```

static 블록을 사용하여 초기화 작업이 가능하다.

예>

```
public class 클래스명{

    static{
        // 초기화 작업 처리
    }
}
```

7장. 클래스의 관계

1. 클래스의 관계
- 2-3. has a , is a 관계
- 4-5. 상속(inheritance)
6. super 키워드
7. 접근 지정자
8. 메서드 오버라이딩(overriding)
9. 다형성 (polymorphism)
10. Object 클래스

1. 클래스의 관계 (Class Relationship)

현실세계에서는 객체간에 특별한 관계를 가지고 존재하게 된다.

회사에서는 상사와 부하직원 관계, 학교에서는 선생님과 친구관계, 가정에서는 부모와 자식 관계 등으로 맺어져 있다.

가상세계에서도 클래스간에 특별한 관계를 맺고 있으며 대표적으로 has a 와 is a 관계가 있다.

가. has a 관계

한 객체가 다른 객체를 포함하는 관계이다. 전체(whole)와 부분(part)로 구성된다.

```
예> 트럭 has a 엔진  
    트럭 has a 라디오  
    학생 has a 컴퓨터
```

나. is a 관계

같은 종(species)의 객체를 큰 개념과 작은 개념으로 설정한 관계이다.

```
예> 대학생 is a 학생  
    관리자 is a 사원
```

2. has a 관계

JDK 21

가. 집합관계 (aggregation relationship)

whole과 part간의 LifeCycle이 서로 다르다.

예> 트럭 has a 라디오.



나. 구성관계 (composition relationship)

Whole과 part간의 LifeCycle이 서로 일치한다.

예> 트럭 has a 엔진



개념

같은 종의 비슷한 속성 및 동작을 가진 객체들 간의 관계이다.
예를 들어 대학생, 중학생, 초등학생 객체들은 모두 공통된 개념인 학생 객체들이다.
개발자, 부장, 과장, 비서는 모두 공통된 개념인 직원 객체들이다.
비슷한 개념들의 객체들은 모두 공통된 속성 및 동작을 가지고 있으며 이는 결국 공통점들이 중복되어 있다는 것이다.
따라서 공통점들을 추출해서 상위개념의 객체로 만들고 하위 객체들은 상속 받아서 사용하게 하면 중복이 제거되고 재사용성도 향상될 수 있다.
이것이 자바의 상속(inheritance) 개념이다.

특징

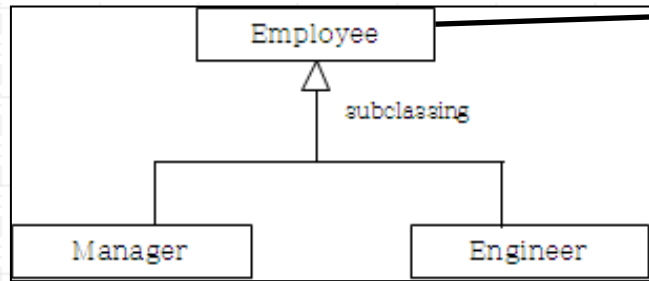
- 객체 간에 is a 관계가 성립된다.
- 하위 클래스에서는 부모 클래스의 멤버(인스턴스 변수/메서드)를 선언 없이 사용 가능하다.
- 부모 생성자와 private로 지정된 부모 멤버는 상속이 안된다.
- 단일 상속 (single inheritance) 만을 지원한다.
- java.lang.Object 클래스가 모든 클래스의 최상위 클래스이다.
- 자바의 모든 클래스는 계층 구조인 상속 관계로 되어있다.

4. 상속 (inheritance)

JDK 21

문법

```
public class SubClasss extends SuperClass {}
```



부모클래스,
상위클래스,
Parent클래스

자식클래스,
하위클래스,
child클래스

예>

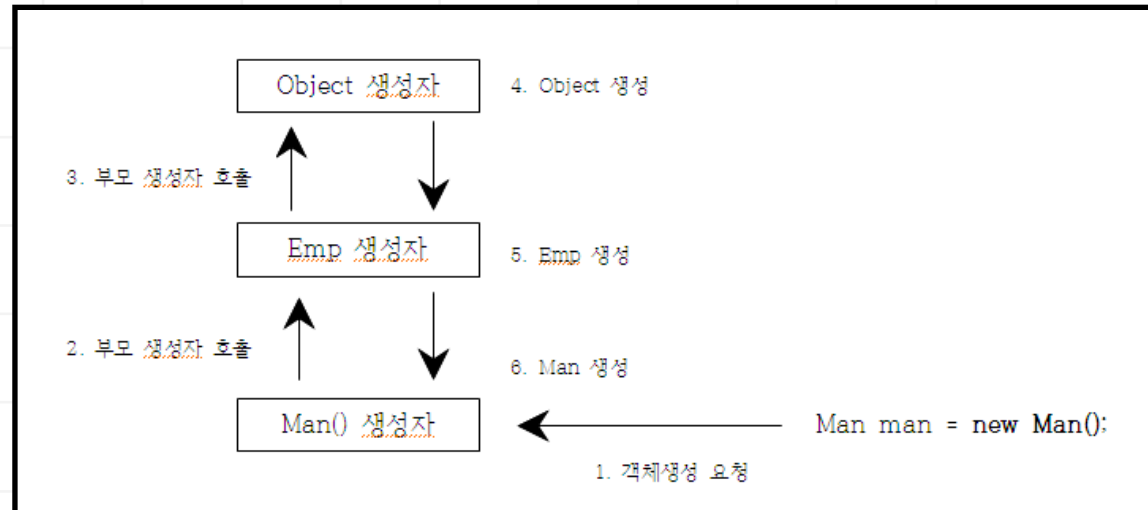
```
public class Employee{} // public class Employee extends Object{} 동일
public class Manager extends Employee{}
public class Engineer extends Employee{}
```

상속관계에서의 생성자 특징

- 생성자는 상속되지 않는다.
- 모든 클래스는 항상 부모 객체를 먼저 생성한 후에 자신이 생성된다.
따라서 객체생성시 항상 Object 클래스가 가장 먼저 생성되고 나머지 클래스들이 생성된다. 이러한 동작 메커니즘은 자식 생성자 첫 라인에서 부모생성자를 호출하는 코드가 자동으로 삽입이 되기 때문이다. (super () 가 자동 삽입됨)
- 필요에 의해서 명시적으로 부모 생성자를 호출할 수도 있는데 반드시 자식 생성자 첫 라인에서 호출해야 된다

예> 부모 생성자 호출 코드

```
super();  
super("홍길동");  
super("홍길동", 20 );
```



개념

super 키워드는 부모의 인스턴스를 가리킨다.
(this 키워드는 자신의 인스턴스를 가리킨다.)

용도

가. 부모 클래스의 멤버와 자식클래스의 멤버가 이름이 동일한 경우
나. 자식클래스에서 명시적으로 부모 생성자를 호출하는 경우
부모에서 선언된 변수인 경우에는 자식에서 초기화하지 않고 super를
이용하여 부모에게 초기화하도록 유도할 수 있다. (권장)

7. 메서드 오버라이딩(overriding)

JDK 21

개념

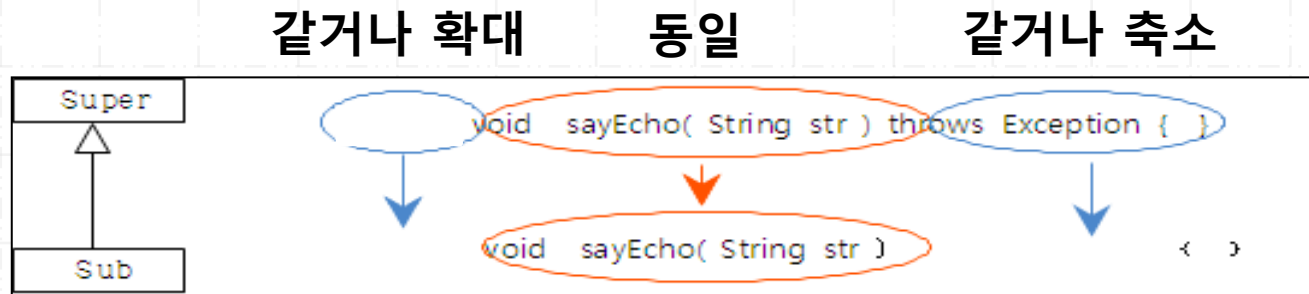
부모클래스의 메서드를 자식이 입맛에 맞게 수정할 수 있다.
이렇게 부모클래스의 메서드를 자식이 수정한 메서드를 오버라이딩 메서드(overriding method)라고 한다

규칙

- 상속 전제
- 메서드 이름이 반드시 동일
- 메서드 리턴 타입이 반드시 동일
- 메서드 인자 리스트가 반드시 동일
- 접근 지정자는 계층구조가 같거나 확대만 가능
- 예외 클래스는 계층구조가 같거나 축소만 가능
- static , final, private로 지정된 메서드는 overriding 불가

7. 메서드 오버라이딩(overriding)

JDK 21



- 오버로딩 메서드(overloading method)
같은 클래스내에 동일한 이름의 메서드가 여러 개 존재 가능.
규칙은 반드시 인자 리스트(개수, 순서, 타입)가 달라야 된다.

8. 다형성(polymorphism) 개념

JDK 21

개념

하나의 참조변수가 여러 다른 데이터형을 참조할 수 있는 능력을 의미한다.
반드시 상속이 전제되어야 하고 필요시 형변환도 가능하다.

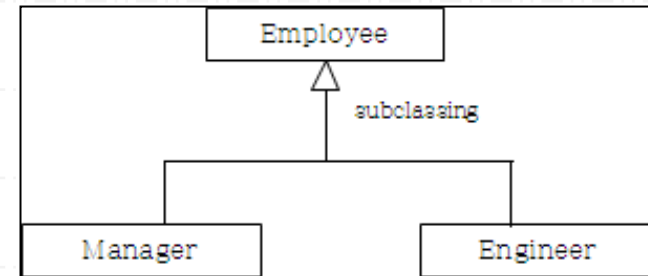
예>

// 다형성 미적용

```
Employee e = new Employee();  
Manager m = new Manager();  
Engineer eng = new Engineer();
```

//다형성 적용

```
Employee emp = new Employee();  
emp = new Manager();  
emp = new Engineer();
```



8. 다형성(polymorphism) 개념

JDK 21

기본형 데이터 (정수형)

long 큰 타입
int ↑
short
byte 작은 타입

예>

```
int n = 10;  
long n2 = n;
```

참조형 데이터 (클래스)

Employee 큰 타입
 ↑
Manager Engineer 작은 타입

예>

```
Manager m = new Manger();  
Employee e = m;
```

적용1: 배열 사용

일반적으로 배열은 같은 타입만을 저장할 수 있다. 하지만 다형성을 적용시키면 다른 타입도 저장 가능하다.

따라서 Object 배열은 자바의 모든 데이터를 저장할 수 있게 된다.

예 >

```
Object [] obj = { "홍길동", new Person(), new Employee() };
```

```
Employee [] emp = { new Manager("홍길동"),  
                    new Manager("이순신"),  
                    new Engineer("유관순") };
```

```
for( Employee e : emp ){  
    if( e instanceof Manager){  
        Manager m = (Manager)e;  
    }  
}
```

적용2: 메서드 파라미터

메서드 파라미터로 다형성을 적용시키면 메서드 호출할 때 자식 타입도 저장 가능하다. 따라서 오버로딩 메서드를 사용하지 않아도 된다.

예 >

```
public class Employee{  
    public void taxRate( Employee emp){  
  
        if( emp instanceof Manager){  
            Manager m = (Manager)emp;  
            // 20%  
        }else if( emp instanceof Engineer ){  
            Engineer eng = (Engineer)emp;  
            // 5%  
        }else if ( emp instanceof Employee){  
            // 10%  
        }  
    }  
}
```

Manager m = new Manager();
taxRate(m);

Engineer eng = new Engineer();
taxRate(eng);

개념

모든 클래스의 최상위 클래스이다.
명시적으로 extends 하지 않아도 자동으로 상속받는다.

가. equals 메서드

동등 비교시 기본 데이터는 == 연산자로 비교하면 된다.
하지만 참조형의 인스턴스는 반드시 equals 메서드로 비교해야 된다.

예>

```
int num = 3;  
int num2 = 3;  
  
if( num == num2 ){}
```

예>

```
Manager m  = new Manager("홍길동");  
Manager m2 = new Manager("홍길동");  
  
If( m.equals(m2) ){ }    // 실제값 비교  
  
If( m == m2 ) { }       //주소값 비교
```

예>

```
String name  = "홍길동";  
String name2 = "홍길동";  
  
if( name.equals( name2 ) ){ }
```

나. toString 메서드

참조변수가 참조하는 인스턴스의 주소값을 문자열로 리턴시키는 메서드로서
참조변수 값을 콘솔에 출력하기 위해서 println(참조변수) 할 때 자동으로 호출된다.

예>

```
Person p = new Person( "홍길동");  
System.out.println( p ); // p.toString() 동일
```

개념 및 은닉화 (캡슐화)

클래스들 간의 접근을 제어하는 용도로 사용한다.

접근 지정자를 사용함으로써 올바른 데이터를 설정할 수 있고 또한 접근을 제한함으로써 접근 대상에 대한 복잡성을 감소 시킬 수도 있다.

접근 지정자	같은 클래스	같은 패키지	상속 관계	다른 패키지
public	가능	가능	가능	가능
protected	가능	가능	가능	불가
(default)	가능	가능	불가	불가
private	가능	불가	불가	불가

8장. 추상 클래스와 인터페이스

중첩 클래스

1. 추상클래스와 인터페이스 개요
2. 추상클래스
3. 인터페이스
4. 디커플링(decoupling)
5. 중첩 클래스

앞에서 배운 상속은 객체지향 프로그래밍의 핵심기능으로서 상속을 적용하면 코드의 재사용 및 다형성, 오버라이딩 메서드와 같은 객체지향적인 프로그램 기법을 적용할 수 있다. 하지만 강력한 상속을 적용시켜도 하위 클래스에서 부모의 메서드를 상속받아서 사용하지 않고 자신만의 메서드를 작성하여 사용한다면 상속을 사용하는 장점을 얻을 수 없다.

상속은 강제성이 없기 때문이다.

따라서 객체지향 특징인 재사용성 및 유지보수를 향상시키기 위해서 하위 클래스에서 반드시 부모 클래스의 메서드를 사용하게끔 강제할 필요성이 요구하게 되며 자바에서는 인터페이스와 추상 클래스를 통해서 하위 클래스들에게 부모의 메서드를 반드시 사용하도록 강제 할 수 있다.

강제를 함으로써 통일성 및 일관성이 지켜질 수 있으며 결국에는 재사용성 및 유지보수가 향상되고 관리하기도 쉬워진다. 인터페이스란 용어자체가 '창구 역할'을 하는 것을 지칭하기 때문에 여러 창구(메서드)가 존재하는 것보다는 통일된 창구(부모에서 정의된 메서드)를 통해서 의사 소통하는 것이 더욱 효율적인 방법이다.

2. 추상 클래스 (abstract class)

JDK 21

개념

body(중괄호)가 없는 추상 메서드를 포함 할 수 있고 abstract로 지정된 클래스를 추상클래스라고 한다.

추상 메서드 문법

```
public abstract 리턴타입 메서드명([인자]);
```

추상 클래스 문법

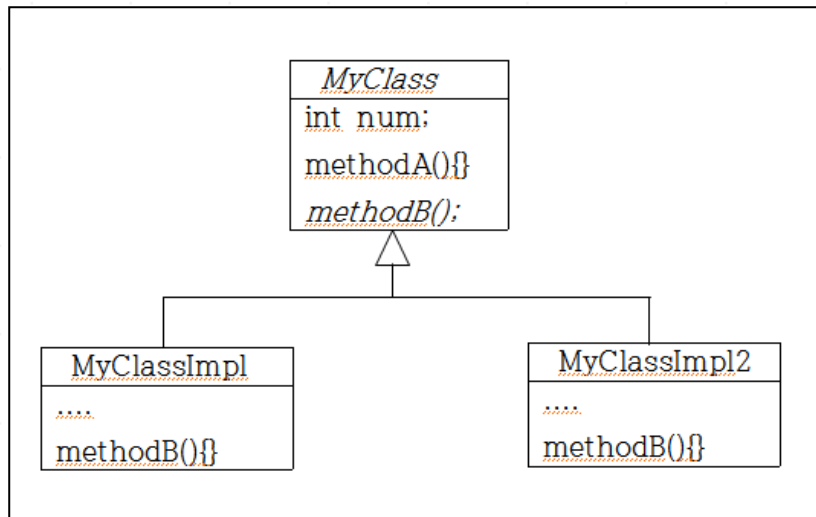
```
public abstract class 클래스명{  
    // 인스턴스 변수  
    // concrete 메서드( 메서드 )  
    // 생성자  
    // 추상메서드 ( abstract method )  
}
```

2. 추상 클래스 (abstract class)

JDK 21

특징

- 추상 메서드를 포함할 수 있기 때문에 미완성 클래스이다.
- 추상 클래스는 직접적인 객체생성이 불가능하다. 추상 클래스의 요소를 사용하기 위해서는 일반 클래스를 이용하여 추상 메서드를 overriding 하여 사용된다.
- 추상 클래스도 클래스이기 때문에 단일상속만 지원된다.
- 강제성 및 통일성을 제공한다. 하위 클래스에게 상속의 장점인 부모의 변수와 메서드를 선언 없이 사용할 수도 있고 또한 추상 메서드를 이용한 특정 메서드만 강제할 수도 있다.
- `abstract`는 UML 표기법으로 이탤릭체를 사용한다.
- `new`는 불가능하지만 다형성에 의한 변수의 데이터 형으로 사용 가능하다.



큰 타입

작은 타입

3. 인터페이스 (interface)

JDK 21

개념

상수, 추상 메서드, default 메서드, static 메서드를 포함 할 수 있고 interface 키워드를 사용하여 작성한다.

인터페이스 문법

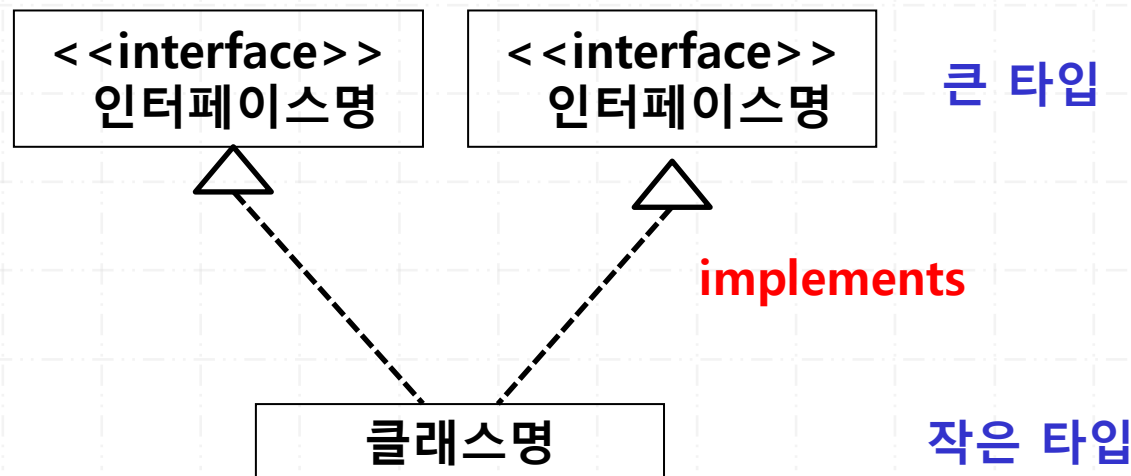
```
public interface 인터페이스명{  
    // 상수  
    // 추상 메서드  
    // default 메서드  
    // static 메서드  
}
```

3. 인터페이스 (interface)

JDK 21

특징

- 상수는 `public static final` 이 자동으로 지정된다.
- 추상 메서드는 `public abstract` 가 자동으로 지정된다.
- 객체생성이 불가능하기 때문에 독자적으로 사용 못하고 일반 클래스를 이용해서 추상 메서드를 overriding 해야 된다
- implements 키워드를 이용한다.
- 다중 구현이 가능하고 인터페이스 간에는 다중 상속이 가능하다.
- 강제성과 통일성 확보 목적으로 사용되고 다형성에 의한 변수의 데이터 형으로 사용 가능하다.



3. 인터페이스 (interface)

JDK 21

예 >

```
public interface Flyer{
    public void takeOff();
    public abstract void fly();
    public void land();
}

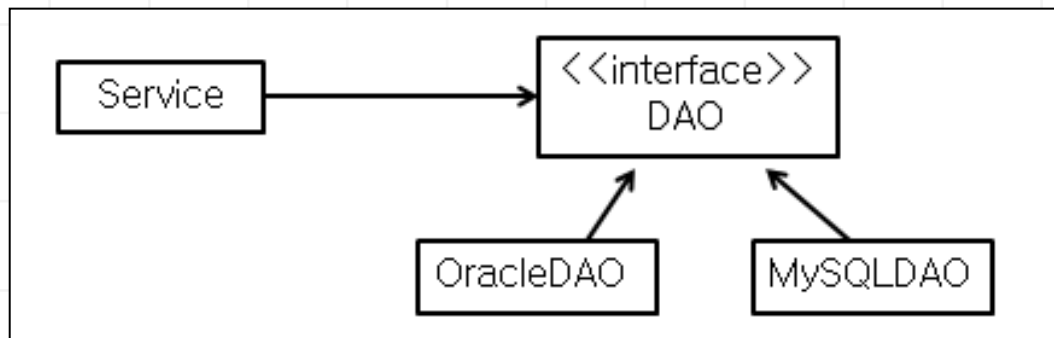
public class Bird implements Flyer {
    @Override
    public void takeOff(){}
    @Override
    public void fly(){}
    @Override
    public void land(){}
}

Flyer f = new Bird();    // 다형성
f.takeOff();
..
```

4. 디커플링 (decoupling)

JDK 21

- 인터페이스를 이용한 디커플링(decoupling)
 - 커플링이란 한쌍을 일컫는 말이다. 동사로는 '두 대상을 결합한다'라는 의미로 사용된다.
 - 프로그래밍에서 디커플링이란 '모듈 사이의 연결고리(의존 관계)를 인터페이스를 사용하여 두 모듈의 의존성을 감소하여 개발 및 유지보수가 용이하도록 처리하는 방법'을 의미한다.
 - 즉, 모듈 사이를 인터페이스로 연결하고 구현체를 분리하는 것을 말한다.
 - 객체들 간에 결속력이 강하면 그 프로그램의 구성 요소 중 하나를 수정하게 되면 그와 연관된 모든 구성 요소를 새로 수정해야 된다. 그러나 인터페이스를 이용한 설계가 잘된 경우에는 수정하고자 하는 요소만 수정하여 재배포 하면 된다.



5. 중첩 클래스 (nested class)

JDK 21

개념

클래스 안에 또 다른 클래스가 정의될 수 있다.
이런 형태의 클래스를 '중첩 클래스' 라고 한다.

기본 구조

```
public class Outer{  
    class Inner{  
  
    }//end Inner  
  
}//end Outer
```

5. 중첩 클래스 (nested class)

JDK 21

특징

- Inner 클래스는 Outer 클래스의 멤버처럼 동작된다.
즉, Outer 클래스가 생성되어야 Inner 클래스를 사용할 수 있다.
결국 Outer 클래스를 통해서 Inner 클래스에 접근할 수 있다는 것이다.
- Inner 클래스는 Outer 클래스의 멤버(변수/메서드)를 마치 자신의 멤버처럼 사용 가능하다.
(private 도 접근 가능)
- Inner 클래스 안에는 static 변수를 사용할 수 없다. 단, static Inner 클래스에서는 사용 가능하다.
- 소스파일을 컴파일 하면 Outer\$Inner.class 형식으로 클래스 파일이 생성된다.
- Inner 클래스가 Outer 클래스 안에 정의되는 위치에 따라서 4가지 종류가 제공된다.

종류	설명
member	Outer 클래스의 멤버 변수나 메서드처럼 클래스가 정의된 경우이다.
local	Outer 클래스의 특정 메서드 안에서 클래스가 정의된 경우이다.
static	static 키워드를 이용해서 클래스가 정의된 경우이다.
anonymous	익명 클래스를 이용해서 클래스가 정의된 경우이다.

개념

Outer 클래스의 멤버처럼 클래스가 정의된 경우이다.
즉, 객체를 생성해야만 사용할 수 있는 멤버들과 같은 형태이기 때문에 반드시 Outer 클래스를 객체생성 후에 Inner 클래스를 사용할 수 있다.

문법

```
public class Outer {  
    ..  
    class Inner{  
        ...  
    }  
}
```

사용방법

```
// main 메서드내  
Outer xxx = new Outer();  
Outer.Inner yyy = xxx.new Inner();
```

- Outer 클래스의 메서드 안에서 정의된 경우이다.
- 메서드 안에서 정의 되었기 때문에 로컬 변수처럼 동작된다.
- 메서드가 호출 될 때 생성되고 메서드가 종료 될 때 삭제된다.
- Inner 클래스의 객체 생성은 outerMethod 메서드 안에서 한다.

```
public class Outer {  
    public void outerMethod(){  
        int x = 10; // 상수화  
        class Inner{  
            //end Inner  
            Inner I = new Inner();  
        }//end outerMethod  
    }//end Outer
```

```
// main 메서드내
Outer outer = new Outer();
outer.outterMethod();
```

개념

static 을 이용하여 inner 클래스를 정의한 경우이다.
일반 Inner클래스는 static 변수를 포함할 수 없지만 static Inner 클래스는 가능하다.
반면에 Outer 클래스의 멤버변수는 접근이 불가능하다.

문법

```
public class Outer {  
    int x = 5;  
    static class Inner{  
        static int num= 10; // 가능  
        ...  
    }//end Inner  
}//end Outer
```

사용방법

```
Outer.Inner yyy = new Outer.Inner();
```

개념

인터페이스 및 추상 클래스 사용시 명시적으로 하위 클래스명을 지정하지 않고 사용하는 방식이다.

문법

문법:

```
인터페이스 변수명 = new 인터페이스(){  
    추상메서드 재정의  
};  
  
변수명.메서드()
```

예 > Flyer 인터페이스

```
public interface Flyer{  
    public void fly();  
}
```

```
Flyer f = new Flyer() {  
    @Override  
    public void fly() {  
        ...  
    }  
};  
f.fly();
```

9장. 유틸리티 클래스

1. String 클래스
2. StringBuffer 클래스
3. Wrapper 클래스
4. Random 클래스
5. 날짜 데이터
6. SimpleDateFormat 클래스
7. LocalTime, LocalDate 클래스
8. StringTokenizer 클래스

String 문자열은 한번 생성하면 변경되지 않는다. (immutability)

1) 리터럴 이용

동일한 문자열인 경우에는 새로 생성하지 않고 재사용한다.
따라서 name1 과 name2의 위치값은 동일하다.

예>

```
String name1 = "홍길동";  
String name2 = "홍길동";
```

2) new 이용

동일한 문자열일지라도 매번 새롭게 생성된다.
따라서 s1 과 s2의 위치값은 서로 다르다.

예>

```
String s1 = new String("hello");  
String s2 = new String("hello");
```

2. String 클래스 메서드

JDK 21

핵심 메서드

메서드	설명
length	문자열의 길이를 반환한다.
equals	문자열이 일치하는지를 검사한다.
equalsIgnoreCase	대소문자 구별없이 문자열이 일치하는지를 검사한다.
substring	부분열을 구한다.
replace	문자열을 대치한다.
toUpperCase	문자열을 대문자로 바꾼다.
toLowerCase	문자열을 소문자로 바꾼다.
charAt	특정위치에 해당하는 문자를 반환한다.
trim, strip	공백문자를 제거한다.
concat	문자열을 연결한다.

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html>

StringBuilder 및 StringBuffer 는 동적으로 크기가 변경된다.

생성 방법

예 >

```
StringBuilder s = new StringBuilder("hello");  
StringBuffer s2 = new StringBuffer("hello");
```

핵심 메서드

메서드	설명
length	문자열의 길이를 반환한다.
capacity	초기 버퍼크기를 지정한다.
append	버퍼에 문자열을 추가한다.
insert/delete	버퍼에 문자열을 삽입/삭제한다.

개념

8개의 기본 데이터에 해당하는 8개의 클래스 파일들을 통칭해서 부르는 이름.

기본데이터	wrapper 클래스
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

1) 오토박싱(auto boxing)

자동으로 기본형 데이터가 wrapper 로 형변환 되는 것을 의미한다.

예>

```
int n = 10;  
Integer n2 = n;
```

2) 언박싱(unboxing)

자동으로 wrapper 클래스가 기본형 데이터로 형변환 되는 것을 의미한다.

예>

```
Integer n = new Integer( 100 );  
int n2 = n;
```

개념

임의의 랜덤값을 발생시키는 유틸리티 클래스이다.

메서드

메서드	설명
nextInt	임의의 정수값을 구한다.
nextInt(범위)	0부터 입력범위-1까지의 임의의 정수값을 구한다.
nextBoolean	임의의 불린값을 구한다.
nextFloat	0.0과 1.0 사이의 임의의 float 값을 구한다.
nextDouble	0.0과 1.0 사이의 임의의 double 값을 구한다.

예 >

```
import java.util.Random;
```

```
Random ran = new Random();
```

```
int n = ran.nextInt(11);
```

1) java.util.Date

예 >

```
import java.util.Date;  
Date d = new Date();  
System.out.println( d );
```

2) java.util.Calendar

예 >

```
import java.util.Calendar;  
  
Calendar c = Calendar.getInstance();  
int year = c.get(Calendar.YEAR);  
int month = c.get(Calendar.MONTH) + 1;  
int day = c.get(Calendar.DAY_OF_MONTH);
```

3) java.time.LocalDate

예>

```
// 현재 날짜
LocalDate today = LocalDate.now();
today.plusDays(1);
today.minusDays(2);

// 특정 날짜
LocalDate xxx = LocalDate.of(2024, 2, 6);
LocalDate xxx2 = LocalDate.parse("2025년02월06일",
    DateTimeFormatter.ofPattern("yyyy년MM월dd일"));
```

개념

Date 클래스를 이용한 날짜 데이터 값을 특정 포맷으로 출력할 때 사용되는 유틸리티 클래스이다.

형식 문자

형식 문자	기능
y	년도 표시
M	월 표시
d	일 표시
H	시간(0~23)표시
m	분 표시
s	초 표시
h	시간(1~12)표시
a	AM/PM 표시

예>

```
import java.text.SimpleDateFormat;

SimpleDateFormat sdf =
    new SimpleDateFormat( "yyyy/MM/dd" );
String str = sdf.format( new Date() );
```

개념

열거형은 변수가 미리 정의된 상수 집합이 될 수 있도록 하는 데이터 유형이다.
반드시 미리 정의된 값 중 하나와 일치해야 된다.

예> 방향 (NORTH, SOUTH, EAST, WEST) , 요일 (MONDAY, ...)

Enum API

<<Java Class>>  Enum<E> java.lang	
 name: String	
 ordinal: int	
 name():String	
 ordinal():int	
 Enum(String,int)	
 toString():String	
 equals(Object):boolean	
 hashCode():int	
 clone():Object	
 compareTo(E):int	
 getDeclaringClass():Class<E>	
 <u>valueOf(Class<T>,String):T</u>	
 finalize():void	
 readObject(ObjectInputStream):void	
 readObjectNoData():void	
 compareTo(Object):int	

```
public enum Day {  
    SUNDAY, MONDAY, TUESDAY, WEDNESDAY,  
    THURSDAY, FRIDAY, SATURDAY;  
}
```

```
public class SimpleEnumExample {  
    enum Days{  
        SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY;  
    }  
}
```

10장. 예외 처리

- 1-2. 예외(exception) 및 클래스
- 3. 예외 종류 2가지 (compile checked, compile unchecked)
- 4. 예외처리 방법 2가지 (try~catch, throws)
- 5-6. 다중 catch문, finally
- 7. throws 예외처리
- 8. throw 명시적 예외 강제 발생
- 9. 사용자 정의 예외 클래스

개요

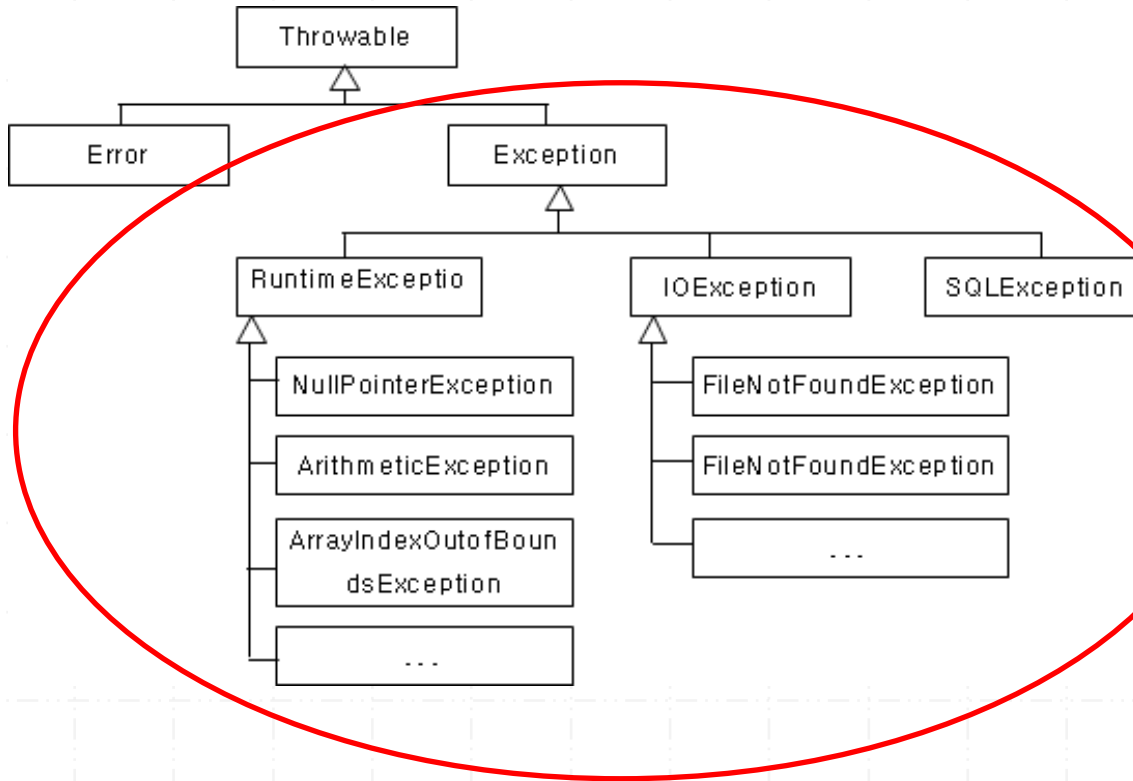
프로그램 실행 중에 발생하는 의도하지 않은 문제 발생을 의미한다.
예외가 발생되면 프로그램은 **비정상종료** 된다.

예외처리

예외발생시 비정상 종료 되는 프로그램을 **정상종료**로 처리하는 작업을 의미한다.
예외처리를 담당하는 예외클래스를 활용하며 최대 예외처리 방법은 다음과 같다.

- 가. 최대한 자세하게 end-user에게 예외가 발생한 이유를 출력한다.
- 나. 최대한 end-user가 이해하기 쉬운 언어로 출력한다.

예외가 발생한 코드를 수정하는 것이 예외처리가 아니다.
코드는 순차문이기 때문에 한번 실행된 문장은 다시 실행 시켜 수정할 수 없다.



일반적으로 Exception 클래스를 예외 클래스의 최상위 클래스로 간주한다.

1) compile checked 예외

- 컴파일시 예외처리 여부를 컴파일러가 체크하여 예외처리가 안되어 있으면 컴파일 에러가 발생된다.
- IOException 과 SQLException 계열에 해당된다.
이것은 자바I/O 및 데이터베이스 관련 작업을 수행하는 메서드를 사용하기 위해서 반드시 예외처리를 해야 된다는 의미이다.

2) compile unchecked 예외

- 컴파일시 예외처리 여부를 컴파일러가 체크하지 않는다.
컴파일러가 체크하지 않는 이유는 개발자가 조건문 코드를 추가 작성하면 발생되지 않을 예외이기 때문이다.
- RuntimeException 계열에 해당되고 NullPointerException, ArithmeticException, ArrayIndexOutOfBoundsException, ClassCastException 등이 있다.

1) try~ catch 문 사용

예외가 발생한 곳에서 예외를 처리하는 방법이다.

문법

```
try{  
    문장1;  
    문장2;  
}catch(예외클래스 e){  
    예외처리코드;  
}
```

2) throws 문 사용

예외가 발생한 곳이 아닌 호출한 메서드로 예외처리를 위임하는 방식이다.

문법

```
public void method() throws 예외클래스 {}  
public void method2() throws 예외클래스{}
```

try 블록 내에서 실행되는 문장이 여러 개 있는 경우에는 발생하는 예외도 달라질 수 있다.

주의할 점은 반드시 계층구조가 낮은 클래스부터 catch해야 된다.

문법

```
try{  
    문장1;  
    문장2;  
}catch( 예외 클래스명 e){  
    예외처리코드;  
}catch( 예외 클래스명 e){  
    예외처리코드;  
}catch(Exception e){  
    예외처리코드;  
}
```

예외 발생 여부와 상관없이 항상 실행되어야 하는 문장을 지정한다.

문법

```
try{
    문장1;
    문장2;
}catch( 예외 클래스명 e){
    예외처리코드;
}catch( 예외 클래스명 e){
    예외처리코드;
}finally{
    반드시 수행되는 문장
}
```

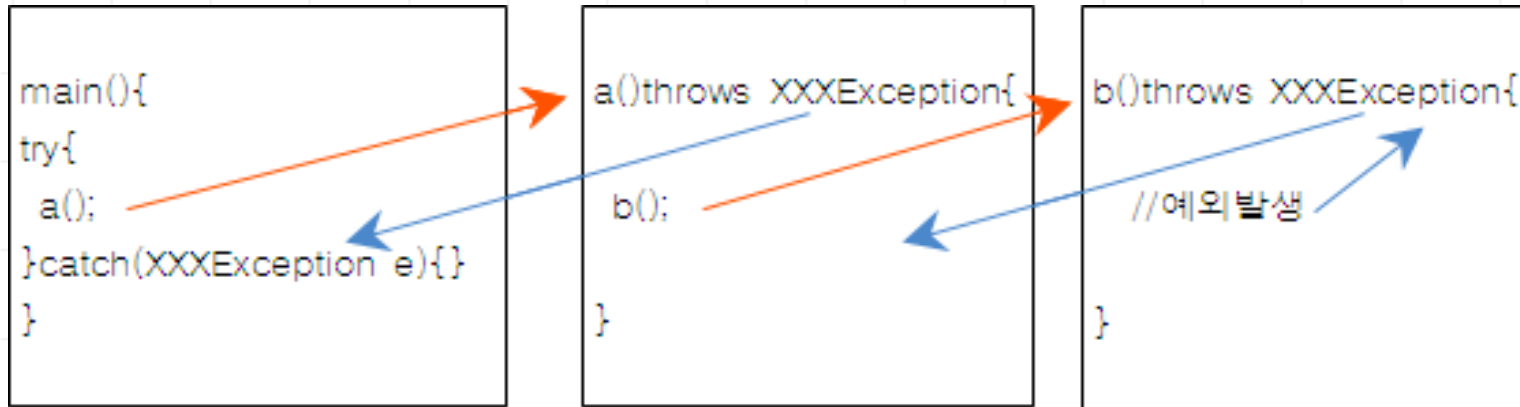
* try ~ finally 문 가능

```
try{
    ...
}finally{
    ...
}
```

예외가 발생되면 호출한 메서드로 예외처리를 위임하는 방식이다.

문법

```
public void method() throws XXXException, XXXException {}
```



throws 문을 이용해서 예외 처리하는 대표적인 경우

- 가. 사용자가 지정한 특정조건에 위배될 경우에 예외를 명시적으로 발생시켜 처리하기 위함이다.
- 나. 발생한 시스템 예외클래스 대신에 사용자가 만든 예외클래스로 처리하기 위함이다.

8. 명시적으로 예외를 강제적으로 발생

JDK 21

특정 조건을 위반했을 경우에 명시적으로 예외를 발생시키는 방법이다.
일반적으로 발생시키는 예외클래스를 직접 구현하여 사용한다.

문법

```
if( 조건 ){  
    throw new XXXException(메시지);  
}
```

예>

```
public void method (int num) throws Exception{  
    if(num < 0){  
        throw new Exception("음수값 에러 발생");  
    }  
}
```

사용자가 만든 어플리케이션에서 특정 조건을 위반했을 경우에 명시적으로 예외를 발생 시킬 수 있다.

이때 발생시킨 예외를 처리할 수 있는 예외클래스는 API에서 제공해주지 못한다.

API는 포괄적인 예외클래스를 제공하며 사용자가 발생시킨 예외는 사용자가 명시적으로 만든 예외 클래스로 처리하는 것이 적합하다.

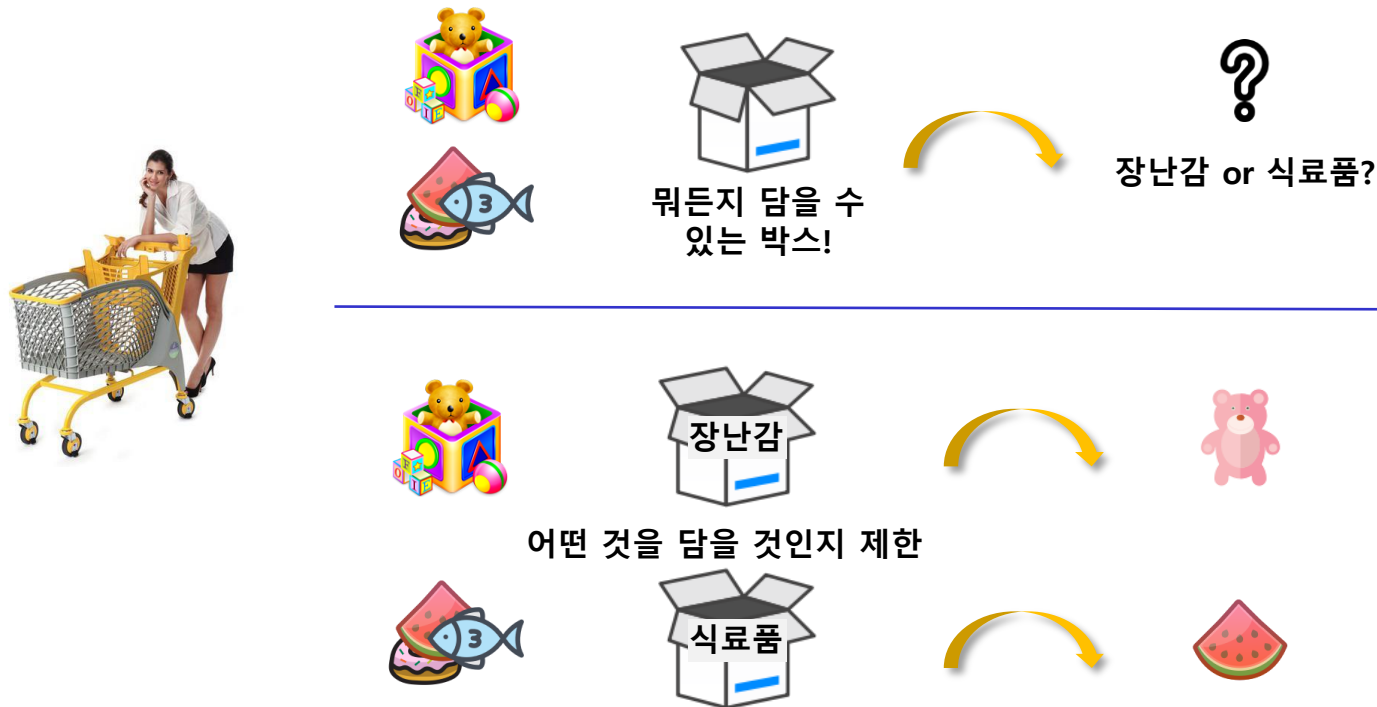
문법

```
public class UserException extends Exception{  
    public UserException(String msg){  
        super(msg);  
    }  
}
```

11장. Generics

1. Generics 개요 및 표현
2. 타입 파라미터 제한
3. Generic method
4. 와일드 카드(?)

- JDK 1.5이후 등장
- 다양한 타입의 객체를 다루는 메서드, 컬렉션 클래스에서 컴파일 시에 타입 체크
 - 미리 사용할 타입을 명시해서 형 변환을 하지 않아도 되게 함
 - 객체의 타입에 대한 안전성 향상 및 형 변환의 번거로움 감소



- 클래스 또는 인터페이스 선언 시 <>에 타입 파라미터 표시

```
public class Class_Name<T>{}  
public interface Interface_Name<T>{}
```

- 타입 파라미터

- 단순히 임의의 참조형 타입을 말하며 그대로 써야 하는 것은 아님
- T : reference Type, E : Element, K : Key, V : Value

```
public class ArrayList<E> extends AbstractList<E>  
    implements List<E>, RandomAccess, Cloneable, java.io.Serializable{... }
```

```
public class HashMap<K,V> extends AbstractMap<K,V>  
    implements Map<K,V>, Cloneable, Serializable {... }
```

- 객체 생성

- 변수 쪽과 생성 쪽의 타입은 같거나 생략 가능.

```
Class_Name<String> generic = new Class_Name<String>();  
Class_Name<String> generic2 = new Class_Name<>();
```

```
Class_Name generic3 = new Class_Name();
```

 Class_Name is a raw type. References to generic type Class_Name<T> should be parameterized

- 컴파일 시점에 타입 파라미터들이 지정된 타입으로 대체됨

```
public class NormalBoxTest {  
  
    public static void main(String[] args) {  
        NormalBox nBox1 = new NormalBox();  
        nBox1.setSome("Hello");  
        nBox1.setSome(new Toy());  
  
        Object some = nBox1.getSome();  
  
        if(some instanceof Toy) {  
            Toy toy = (Toy)some;  
            // toy 사용  
        }else if(some instanceof Grocery) {  
            Grocery grocery = (Grocery)some;  
            // grocery 사용  
        }else {  
            System.out.println("알수 없음");  
        }  
    }  
}
```

Object를 파라미터로 사용 → 어떤 객체든지 수용 가능

```
public class GenericBoxTest {  
  
    public static void main(String[] args) {  
        GenericBox<Toy2> gBox1 = new GenericBox<>();  
  
        gBox1.setSome(new Toy2());  
  
        // gBox1.setSome(new Grocery2());  
  
        Toy2 toy = gBox1.getSome();  
        // toy 사용  
  
        GenericBox<Grocery2> gBox2 = new GenericBox<>();  
        gBox2.setSome(new Grocery2());  
        Grocery2 grocery = gBox2.getSome();  
        // grocery 사용  
    }  
}
```

무언가 T로 객체를 한정 → T의 자식까지만 허용 됨

4. 타입 파라미터 주의할 점

JDK 21

- 타입 파라미터는 instance 멤버로 간주 → static 사용 불가

```
public class Test<I> {  
    static I item; //Cannot make a static reference to the non-static type T  
    static void method(I item) {}  
}
```

- 타입 파라미터를 이용한 객체 생성(new) 및 instanceof 연산자 사용 불가
 - 컴파일 타임에 정확한 객체 타입이 필요한 연산자(new, instanceof)

```
public class Test<I> {  
    I i = new I(); //Cannot instantiate the type I  
    Object obj = new Object();  
    {  
        if(obj instanceof I) {  
            // Cannot perform instanceof check against type parameter I  
        }  
    }  
}
```

5. 타입 파라미터 제한

JDK 21

- 필요에 따라 구체적인 타입 제한 필요
 - 계산기 프로그램 구현 시 Number 이하의 타입(Byte, Short, Integer...)로만 제한
 - type parameter 선언 뒤 extends 와 함께 상위 타입 명시

```
class NumberBox<T extends Number> {  
    public void addSomes(T... ts) {  
        double d = 0;  
        for (T t : ts) {  
            d += t.doubleValue();  
        }  
        System.out.println("총 합은: " + d);  
    }  
}
```



```
public class ExtendsTest {  
  
    public static void main(String[] args) {  
        NumberBox<Number> numBox = new NumberBox<>();  
        numBox.addSomes(1.5, 5, 4L);  
  
        NumberBox<Integer> intBox = new NumberBox<>();  
        intBox.addSomes(1,2,3);  
  
        //NumberBox<String> strBox = new NumberBox<>();  
    }  
}
```

- 인터페이스로 제한할 경우도 extends 사용
- 클래스와 함께 인터페이스 제약 조건을 이용할 경우 & 로 연결

```
class TypeRestrict1<T extends Cloneable>{}  
  
class TypeRestrict2<T extends Number & Cloneable>{}
```

- 파라미터와 리턴타입으로 type parameter를 갖는 메서드
 - 매서드 리턴 타입 앞에 타입 파라미터 변수 선언

```
[제한자] <타입_파라미터, [...]> 리턴_타입 메서드_이름(파라미터){  
    // do something  
}
```

```
public class TypeParameterMethodTest<T> {  
    T some;  
    public TypeParameterMethodTest(T some){  
        this.some = some;  
    }  
    public <P> void method1(P p) {  
        System.out.println("클래스 레벨의 T" + some.getClass().getName());  
        System.out.println("파라미터: " + p.getClass().getName());  
    }  
  
    public <P> P method2(P p) {  
        return p;  
    }  
  
    public static void main(String[] args) {  
        TypeParameterMethodTest<String> tpmt = new TypeParameterMethodTest<>("Hello");  
        tpmt.method1(10);  
        tpmt.<Long>method2(20L);  
    }  
}
```

객체 생성 시점에 T의 타입 결정

메서드 호출 시점에 P의 타입 결정

7. 와일드 카드(?)

- generic type을 매개변수나 리턴 타입으로 사용할 때 구체적인 타입 대신 사용

표현	설명
Generic type<?>	타입에 제한이 없음
Generic type<? extends T>	T 또는 T를 상속받은 타입들만 사용 가능
Generic type<? super T>	T 또는 T의 조상 타입만 사용 가능

```
public class WildTypeTest {  
  
    public void method1(PersonBox<?> some) {}  
    public void method2(PersonBox<? extends Person> some) {}  
    public void method3(PersonBox<? super Person> some) {}  
  
    public static void main(String[] args) {  
        WildTypeTest wtt = new WildTypeTest();  
        wtt.method1(new PersonBox<Object>());  
        wtt.method1(new PersonBox<Person>());  
        wtt.method1(new PersonBox<SpiderMan>());  
  
        //wtt.method2(new PersonBox<Object>());  
        wtt.method2(new PersonBox<Person>());  
        wtt.method2(new PersonBox<SpiderMan>());  
  
        wtt.method3(new PersonBox<Object>());  
        wtt.method3(new PersonBox<Person>());  
        //wtt.method3(new PersonBox<SpiderMan>());  
    }  
}
```

12장. 컬렉션 API

1. 컬렉션 API
2. 컬렉션 API 계층구조
3. 컬렉션 관련 인터페이스
4. Set 계열
5. List 계열
6. Map 계열

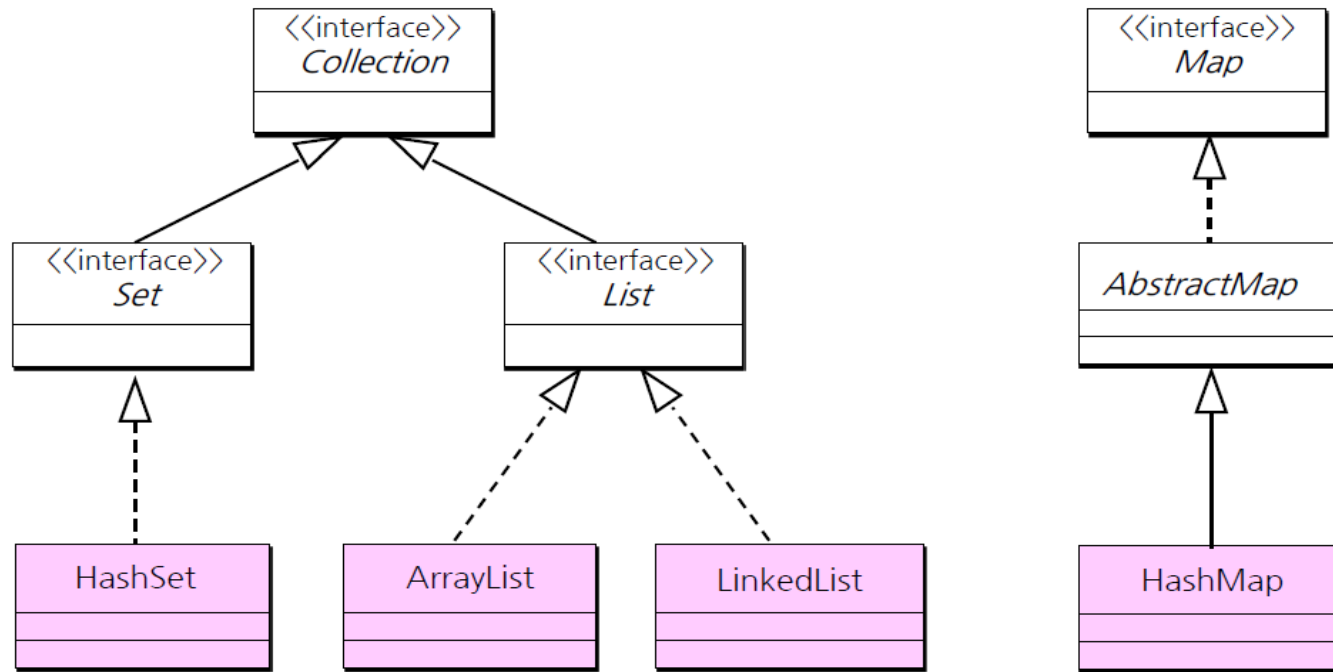
개념

- 다수의 데이터를 쉽게 처리할 수 있는 표준화 된 방법을 제공하는 클래스들.
- 일반적으로 List, Set, Map 의 3가지 타입 API를 제공.

특징

- 객체만 저장 가능하다. 기본형은 wrapper 클래스를 사용한다.
- 객체를 저장할 때마다 자동으로 크기가 늘어난다.
- 저장된 객체를 삭제, 수정이 가능하고 삽입도 가능하다.
- Set 계열 : 순서가 없기 때문에 중복이 허용되지 않는다.
- List 계열: 순서가 있기 때문에 중복이 허용된다.
- Map 계열: 키와 값의 쌍으로 저장되고 순서는 없다.

- 컬렉션 API



❖ Collection 인터페이스

Method	설명
boolean add(Object o) boolean addAll(Collection c)	지정된 객체 또는 Collection의 객체들을 Collection에 추가
void clear()	Collection의 모든 객체를 삭제
boolean isEmpty()	Collection이 비어있는지 확인
int size()	Collection에 저장된 객체의 개수를 반환
Object[] toArray()	Collection에 저장된 객체를 배열로 반환

❖ Map 인터페이스

Method	설명
Object put(Object key, Object value) void putAll(Map m)	Key객체를 id로 value객체를 저장 Map의 모든 key-value를 추가
void clear()	Map의 모든 객체를 삭제
boolean isEmpty()	Map이 비어있는지 확인
int size()	Map에 저장된 객체의 개수를 반환
Set keySet()	Map에 저장된 모든 Key객체를 반환

- Set 계열(순서 없고 중복 저장 불가)

- ❖ HashSet

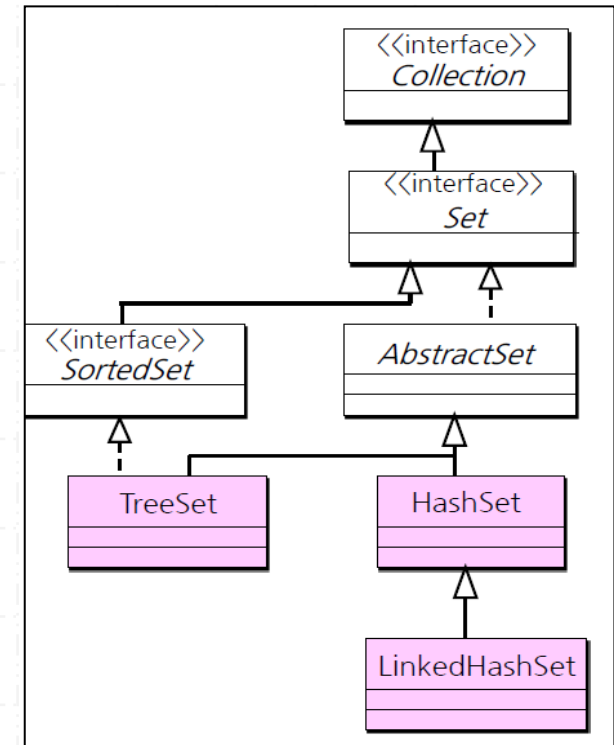
- Set에 객체를 저장하는데 Hash를 사용하여 처리 속도가 빠르다.

- ❖ LinkedHashSet

- HashSet과 거의 같다. 차이점은 Set에 추가되는 순서를 유지한다는 점

- ❖ TreeSet

- 객체의 Hash값에 의한 오름차순의 정렬 유지



- List 계열(순서 있고 중복 저장 가능)

- ❖ List

- List의 요소에는 순서를 가진다.

- ❖ ArrayList

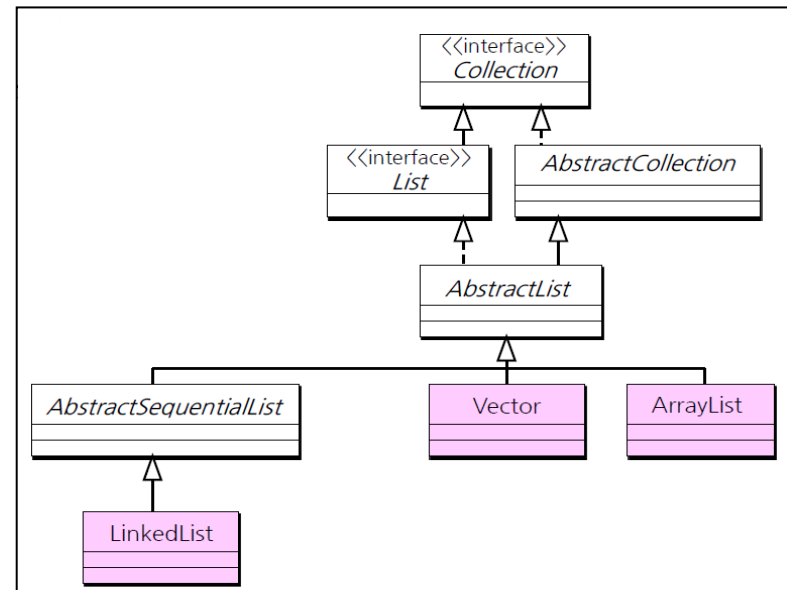
- List에서 객체를 얻어내는데 효율적
- 동기화 (Synchronization) 를 제공하지 않는다.

- ❖ LinkedList

- List에서 앞뒤의 데이터를 삽입하거나 삭제하는데 효율적이다.
- 동기화를 제공하지 않는다.

- ❖ Vector

- 기본적으로 ArrayList와 동등하지만 Vector에서는 동기화를 제공한다.
- 그래서 List객체들 중에서, 가장 성능이 좋지 않다.



- Map 계열(key/value 저장 가능)

- ❖ HashMap

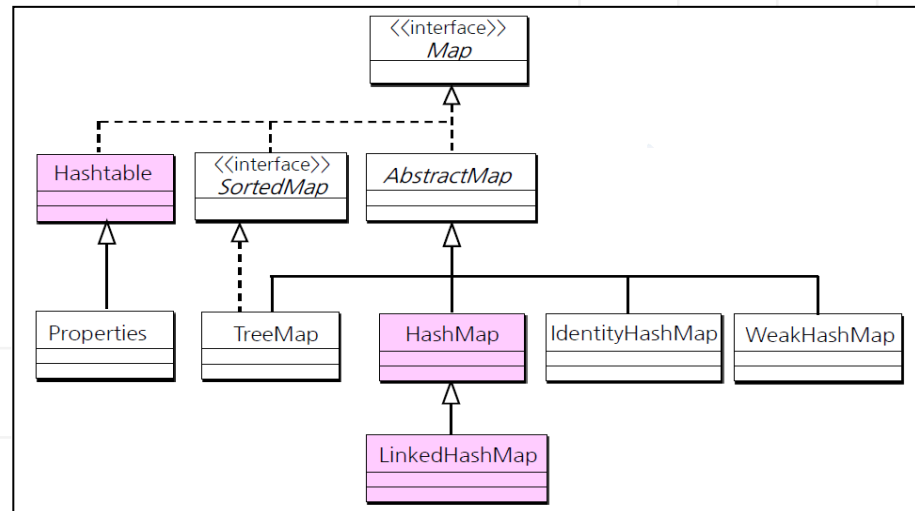
- Map에 키를 저장하는데 hash를 사용하여 성능이 좋다.
- 저장되는 순서가 유지 되지 않는다.
- 오직 하나의 null키를 가질 수 있다.

- ❖ LinkedHashMap

- HashMap과 거의 같다. 차이점은 Map에 추가되는 순서를 유지한다는 점

- ❖ Hashtable

- 동기화(Synchronization)를 제공한다.
- null키와 null 값을 저장할 수 없다.



13장. 람다(lambda) 표현식

1. 람다 개요 및 표현식
2. 함수적 인터페이스
3. 표준 API 함수적 인터페이스

1. 람다(lambda)식이란?

JDK 21

- 함수형 프로그래밍 언어에 적합한 표현식 (병렬처리와 이벤트 처리에 적합)
- 자바 프로그램 = 객체지향 프로그램 + 함수형 프로그램(JDK 1.8)
- 람다 표현식 (Lambda expression)

익명 함수(anonymous function)를 생성하기 위한 식으로서, 객체 지향 언어보다는 함수 지향 언어에 가깝다.

자바 코드가 매우 간결해지고 컬렉션 요소를 필터링 하거나 매핑해서 원하는 결과를 쉽게 집계할 수 있다.

람다식의 형태는 매개변수를 가진 코드 블록이지만 런타임 때에는 익명 구현 객체(익명 클래스)를 생성한다.

```
Runnable runnable = new Runnable() {  
    public void run() { ... }  
};
```

익명 구현 객체

```
Runnable runnable = () -> { ... };
```

람다식

2. 람다(lambda)식 기본 문법

JDK 21

- 함수적 스타일의 람다식 기본문법은 다음과 같다.

```
(타입 매개변수, ...) -> { 실행문; ... }
```

(타입 매개변수,...)는 오른쪽 {} 블록을 실행하기 위해 필요한 값을 제공하는 역할을 한다. 매개변수의 이름은 개발자가 자유롭게 지정 가능하고 -> 기호는 매개 변수를 사용하여 {}를 실행한다는 의미이다.

예를 들어 int 매개변수 a의 값을 콘솔에 출력하는 람다 표현식은 다음과 같다.

하나의 매개변수만 있으면 () 생략이 가능하고, 하나의 실행문만 있다면 {} 생략도 가능하다.

```
(int a) -> { System.out.println(a); }
```



```
(a) -> { System.out.println(a); }
```



```
a -> System.out.println(a)
```

만약 매개변수가 없다면 ()를 생략할 수 없다. 반드시 다음과 같이 ()를 지정해야 된다.

```
() -> { 실행문; ... }
```

리턴문이 있을 경우에는 다음과 같이 return 문으로 결과값을 지정할 수 있다.

구문 이용

```
(x, y) -> { return x + y; };
```

{ }블록에 리턴문만 있을 경우 람다식에서는 return문과 블록을 사용하지 않고 다음과 같이 작성하는 것이 일반적이다.

표현식 이용

```
(x, y) -> x + y
```

3. 함수적 인터페이스 (@FunctionalInterface)

JDK 21

다음과 같이 람다식은 인터페이스와 밀접한 관계가 있다.

```
인터페이스 변수 = 람다식;
```

이 말은 람다식은 인터페이스의 익명 구현 객체(익명 클래스)를 생성한다는 의미이다. 인터페이스는 직접 객체화할 수 없기 때문에 구현 클래스가 필요한데, 람다식은 익명 구현 클래스를 생성하고 객체화 한다.

람다식은 인터페이스의 종류에 따라 작성 방법이 달라지기 때문에 람다식이 대입될 인터페이스를 람다식의 '타겟타입(target type)' 이라고 한다.

- 함수적 인터페이스

람다식은 하나의 메서드를 정의하기 때문에 두 개 이상의 추상 메서드가 선언된 인터페이스는 람다식을 이용하여 구현 객체를 생성할 수 없다.

하나의 추상 메서드가 선언된 인터페이스만이 람다식의 타겟 타입이 될 수 있는데, 이러한 인터페이스를 함수적 인터페이스(functional interface)라고 한다.

@FunctionalInterface 어노테이션은 두 개 이상의 추상 메서드가 선언되지 않도록 도와주는 어노테이션이다.

3. 함수적 인터페이스 (@FunctionalInterface)

JDK 21

```
@FunctionalInterface
```

```
public interface MyFunctionalInterface {  
    public void method();  
    public void otherMethod(); //컴파일 오류  
}
```

1. 매개변수와 리턴값이 없는 형태의 람다식

```
@FunctionalInterface  
interface Flyer{  
    public void fly();  
}
```

```
//1. 일반  
Flyer f = new Flyer() {  
    @Override  
    public void fly() {  
        System.out.println("Fly");  
    }  
};  
f.fly();  
//2. 람다  
Flyer f2 = () -> { System.out.println("Fly2");};  
f2.fly();  
//실행문이 하나면 {} 생략 가능.  
Flyer f3 = () -> System.out.println("Fly3");  
f3.fly();
```

3. 함수적 인터페이스 (@FunctionalInterface)

JDK 21

2. 매개변수가 있는 형태의 람다식

```
@FunctionalInterface
interface Flyer2{
    public void fly(int x);
}
```

```
//1. 일반
Flyer2 f = new Flyer2() {
    @Override
    public void fly(int x) {
        System.out.println("Fly" + x);
    }
};
f.fly(10);

//2. 람다
Flyer2 f2 = (int x) -> { System.out.println("Fly2"+x);};
f2.fly(10);
// 데이터형 생략 가능.
Flyer2 f3 = (x) -> System.out.println("Fly3" + x);
f3.fly(20);
// 매개변수가 하나면 () 생략가능.
Flyer2 f4 = x -> System.out.println("Fly4" + x);
f4.fly(30);

Flyer2 f5 = x -> {    int result = 100 * x;
                    System.out.println("Fly4" + result);
                };
f5.fly(9);
```

3. 함수적 인터페이스 (@FunctionalInterface)

JDK 21

3. 리턴값이 있는 형태의 람다식

```
@FunctionalInterface
interface Flyer3{
    public int fly(int x,int y);
}
```

//1. 일반

```
Flyer3 f = new Flyer3() {
```

```
    @Override
```

```
    public int fly(int x ,int y) {
        System.out.println("Fly");
        return x+y;
    }
```

```
};
```

```
System.out.println(f.fly(10,20));
```

//2. 람다

```
Flyer3 f2 = (int x,int y) -> { System.out.println("Fly2"); return x+y;};
```

```
System.out.println(f2.fly(10,20));
```

// 데이터형 생략 가능.

```
Flyer3 f3 = (x,y) -> {return x+y;};
```

```
System.out.println(f3.fly(10,20));
```

// return문만 있을 경우 {}와 return 생략 가능.

```
Flyer3 f4 = (x,y) -> x+y;
```

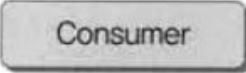
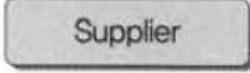
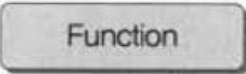
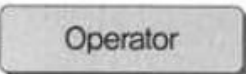
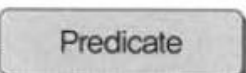
```
System.out.println(f4.fly(10,20));
```

표준 API 함수적 인터페이스

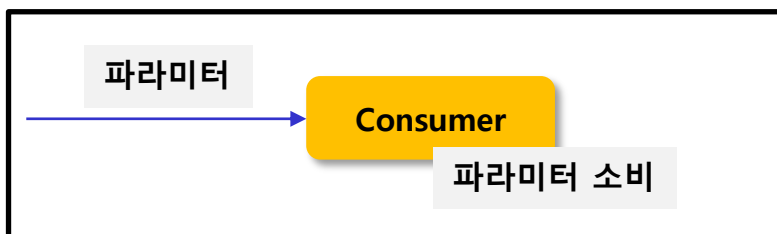
자바 8버전부터 `java.util.function` 패키지내에 유용하게 사용 가능한 함수적 인터페이스를 제공한다.

대표적으로 `Consumer`, `Supplier`, `Function`, `Operator`, `Predicate` 가 지원된다.

구분 기준은 인터페이스에 선언된 추상 메서드의 매개변수와 리턴값의 유무이다.

종류	추상 메소드 특징	
Consumer	- 매개값은 있고, 리턴값은 없음	매개값 → 
Supplier	- 매개값은 없고, 리턴값은 있음	 리턴값 →
Function	- 매개값도 있고, 리턴값도 있음 - 주로 매개값을 리턴값으로 매핑(타입 변환)	매개값 →  리턴값 →
Operator	- 매개값도 있고, 리턴값도 있음 - 주로 매개값을 연산하고 결과를 리턴	매개값 →  리턴값 →
Predicate	- 매개값은 있고, 리턴 타입은 boolean - 매개값을 조사해서 true/false를 리턴	매개값 →  boolean →

기능



1. Consumer

- 매개값은 있고 리턴값은 없다.
- `accept()` 메서드 제공: 단 하나의 매개값만 소비한다.

*Consumer 종류

```
BiConsumer(T,U) : void accept(T t, U u) / 객체 T와 U를 받아서 소비
DoubleConsumer : void accept(double value) / double 값을 받아서 소비
IntConsumer:     void accept(int value) / int 값을 받아서 소비
LongConsumer :   void accept(long value) / long 값을 받아서 소비
ObjDoubleConsumer<T>: void accept(T t, double value) / 객체 T와 double 값을 받아서 소비
ObjIntConsumer<T> : void accept(T t, int value) / 객체 T와 int 값을 받아서 소비
ObjLongConsumer<T> : void accept(T t, long value) / 객체 T와 long 값을 받아서 소비
```

5. Consumer 인터페이스

JDK 21

```
//1. Consumer
Consumer<String> c = a -> {
    System.out.println("param" + a);
};

c.accept("홍길동");
```

```
//2. BiConsumer
BiConsumer<String, Integer> c2 = (a,b)->{
    System.out.println(a+b);
};
```

```
//3. DoubleConsumer
DoubleConsumer c3 = a ->{
    System.out.printf("%.3f\n",a);
};
```

```
c3.accept(3.14);
```

```
//4. IntConsumer
```

```
IntConsumer c4 = a -> {
    System.out.println(a+1);
};
```

```
c4.accept(100);
```

```
//5. LongConsumer
```

```
LongConsumer c5 = a -> {
    System.out.println(a);
};
```

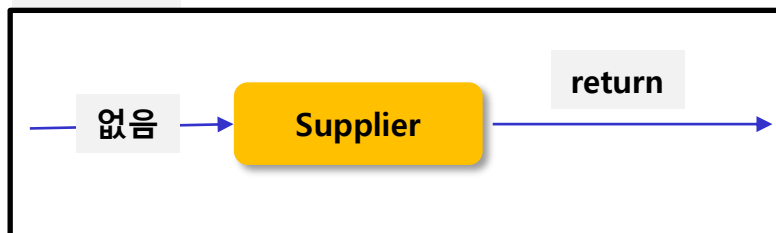
```
c5.accept(100);
```

```
//6. ObjDoubleConsumer
```

```
ObjDoubleConsumer<Person> c6 = (a,b) -> {
    System.out.println(a);
    System.out.println(b);
};
```

```
c6.accept(new Person("홍길동", 20), 3.14);
```

기능



2. Supplier

- 매개변수가 없고 리턴값이 존재
- 실행후에 호출한 곳으로 데이터를 리턴(공급)하는 역할을 담당한다.
- 리턴하는 `getXXX()` 메서드 제공

* 종류

```
Supplier(T)      : T get()           / T 객체를리턴
BooleanSupplier: boolean getAsBoolean( ) / boolean 값을 리턴
DoubleSupplier  : double getAsDouble( ) / double 값을리턴
IntSupplier     : int getAsInt( )      / int 값을리턴
LongSupplier    : long getAsLong( )    / long 값을리턴
```

```
//1. Supplier
Supplier<String> s = () -> {
    return "홍길동";
};

String data = s.get();
System.out.println(data);

Supplier<String> s2 = () -> "홍길동";
System.out.println(s2.get());
```

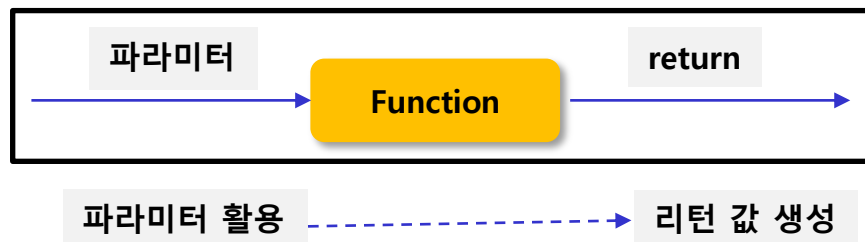
```
//2. BooleanSupplier
int num = 10;
BooleanSupplier s3 = () -> { return (num%2 !=0)?true:false; };
System.out.println(s3.getAsBoolean());

//3. DoubleSupplier
DoubleSupplier s4 = () -> { return 3.14; };
System.out.println(s4.getAsDouble());

//4. IntSupplier
IntSupplier s5 = () -> { return 100; };
System.out.println(s5.getAsInt());

//5. LongSupplier
LongSupplier s6 = () -> { return 1000; };
System.out.println(s6.getAsLong());
```

기능



주로 파라미터를 이용해 연산을 수행한 후 원하는 타입으로 반환하는 역할

3. Function

- 매개변수와 리턴값이 존재
- applyXXX() 메서드 제공

* 종류

```
Function(T,R)           : R apply(T t)          / 객체 T를 객체 R로 매핑
BiFunction(T,U,R)       : R apply(T t, U u)     / 객체 T와 U를 객체 R로 매핑
DoubleFunction(R)       : R apply(double value) / double을 객체 R로 매핑
IntFunction(R)          : R applyInt(value)      / int를 객체 R로 매핑
IntToDoubleFunction     : double applyAsDouble(int value) / int를 double로 매핑
IntToLongFunction       : long applyAsLong(int value) / int를 long으로 매핑
LongToDoubleFunction    : double applyAsDouble(long value) / long을 double로 매핑
LongToIntFunction       : int applyAsInt(long value) / long을 int로 매핑

ToDoubleBiFunction(T, U) : double applyAsDouble(T t, U u) / 객체 T와 U를 double로 매핑
ToDoubleFunction(T)      : double applyAsDouble(T t) / 객체 T를 double로 매핑
ToIntBiFunction<T,U>     : int applyAsInt(T t, U u) / 객체 T와 U를 int로 매핑
ToIntFunction<T>         : int applyAsInt(T t) / 객체 T를 int로 매핑
ToLongBiFunction<T,U>    : long applyAsLong(T t, U u) / 객체 T와 U를 long으로 매핑
ToLongFunction<T>        : long applyAsLong(T t) / 객체 T를 long으로 매핑
```

```
//1. Function(T,R)
Function<String,Integer> f = a -> { return 100;};
int n = f.apply("홍길동");
System.out.println(n);

//2. BiFunction(T,U,R)
BiFunction<String, String, Integer> f2 = (a,b) ->{ System.out.println(a+"\t"+b);
return 100;};

int n2 = f2.apply("홍길동", "이순신");
System.out.println(n2);
```

```
//3. DoubleFunction(R)
DoubleFunction<Integer> f3 = (a) -> { return (int)(a+100);};
int n3 = f3.apply(100);
System.out.println(n3);
```

```
//4. IntFunction(R)
IntFunction<Integer> f4 = a -> { return a+100;};
int n4 = f4.apply(100);
System.out.println(n4);
```

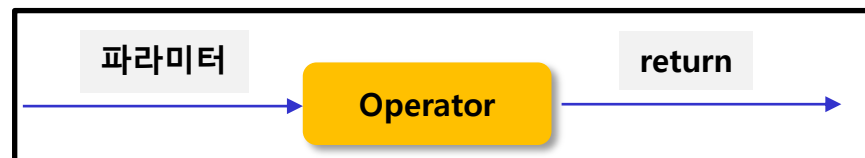
```
//5. IntToDoubleFunction
IntToDoubleFunction f5 = a -> { return a;};
double n5 = f5.applyAsDouble(100);
System.out.println(n5);
```

```
//6. LongToDoubleFunction
LongToDoubleFunction f6 = a -> { return a;};
double n6 = f6.applyAsDouble(100);
System.out.println(n6);

//7. LongToIntFunction
LongToIntFunction f7 = a -> { return (int)a;};
int n7 = f7.applyAsInt(100);
System.out.println(n7);

//8. ToDoubleBiFunction(T, U)
ToDoubleBiFunction<String, String> f8 = (a,b) -> 100;
double n8 = f8.applyAsDouble("홍길동", "이순신");
System.out.println(n8);
```

기능



파라미터 활용 → 같은 파라미터 타입 리턴

주로 파라미터를 이용해 연산을 수행한 후 동일한 타입으로 반환하는 역할

4. Operator

- `Function`과 마찬가지로 매개변수와 리턴값이 존재한다.
하지만 이 메서드들은 매개값을 리턴값으로 매핑하는 역할보다는
매개값을 이용하여 연산을 수행한 후에 동일한 타입으로 리턴값을 제공하는 역할에 중점을 둔다.

*종류

<code>BinaryOperator(T)</code>	: <code>BiFunction (T, U, R)</code> 의 하위 인터페이스	/ T와 U를 연산한 후 R 리턴
<code>UnaryOperator(T)</code>	: <code>Function (T, R)</code> 의 하위 인터페이스	/ T를 연산한 후 R 리턴
<code>DoubleBinaryOperator</code>	: <code>double applyAsDouble(double, double)</code>	/ 두 개의 <code>double</code> 연산
<code>DoubleUnaryOperator</code>	: <code>double applyAsDouble(double)</code>	/ 한 개의 <code>double</code> 연산
<code>IntBinaryOperator</code>	: <code>int applyAsInt(int, int)</code>	/ 두 개의 <code>int</code> 연산
<code>IntUnaryOperator</code>	: <code>int applyAsInt(int)</code>	/ 한 개의 <code>int</code> 연산
<code>LongBinaryOperator</code>	: <code>long applyAsLong(long, long)</code>	/ 두 개의 <code>long</code> 연산
<code>LongUnaryOperator</code>	: <code>long applyAsLong(long)</code>	/ 한 개의 <code>long</code> 연산

```
//1. BinaryOperator(T)
BinaryOperator<String> s = (a,b) ->{ return a.charAt(0)+" "+b.charAt(0);};
String data = s.apply("홍길동", "이순신");
System.out.println(data);

BinaryOperator<Integer> ss = (a,b) ->{ return a+b;};
int xx = ss.apply(100, 200);
System.out.println(xx);
```

```
//2. UnaryOperator(T)
UnaryOperator<String> s2 = a -> { return a.concat("님~~");};
String data2 = s2.apply("홍길동");
System.out.println(data2);
```

```
//3. DoubleBinaryOperator
DoubleBinaryOperator s3 = (a,b) ->{return a+b;};
double data3 = s3.applyAsDouble(3.14, 3.14);
System.out.println(data3);
```

```
//4. DoubleUnaryOperator
DoubleUnaryOperator s4 = a -> a*100;
double data4 = s4.applyAsDouble(3.14);
System.out.println(data4);
```

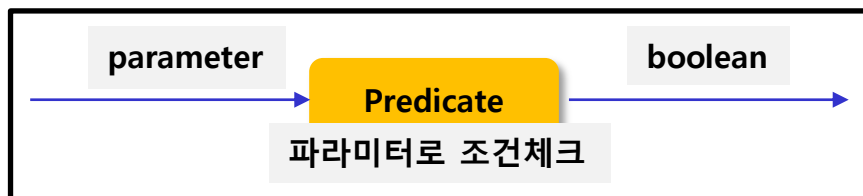
```
//5. IntBinaryOperator
IntBinaryOperator s5 = (a,b) -> a+b;
int data5 = s5.applyAsInt(100, 200);
System.out.println(data5);
```

```
//6. IntUnaryOperator
IntUnaryOperator s6 = a -> a+100;
int data6 = s6.applyAsInt(5);
System.out.println(data6);
```

```
//7. LongBinaryOperator
LongBinaryOperator s7 = (a,b) -> a-b;
long data7 = s7.applyAsLong(200, 50);
System.out.println(data7);
```

```
//8. LongUnaryOperator
LongUnaryOperator s8 = a -> 100-a;
long data8 = s8.applyAsLong(49);
System.out.println(data8 );
```

기능



5. Predicate

- 매개변수와 **boolean** 리턴값이 존재한다.
- **testXXX()**

* 종류

<code>Predicate(T)</code>	:	<code>boolean test(T t)</code>	/ 객체 T를 조사
<code>BiPredicate(T, U)</code>	:	<code>boolean test(T t, U u)</code>	/ 객체 T와 U를 비교조사
<code>DoublePredicate</code>	:	<code>boolean test(double value)</code>	/ <code>double</code> 값을조사
<code>IntPredicate</code>	:	<code>boolean test(<u>int</u> value)</code>	/ <u><code>int</code></u> 값을조사
<code>LongPredicate</code>	:	<code>boolean test(<u>long</u> value)</code>	/ <u><code>long</code></u> 값을조사

```
//1. Predicate(T)
Predicate<String> p = a -> { return a.length()==5;};
boolean x = p.test("hello");
System.out.println(x);

//2. BiPredicate(T, U)
BiPredicate<String, Integer> p2 = (a,b) ->{return true;};
boolean x2 = p2.test("홍길동", 100);
System.out.println(x2);
```

```
//3. DoublePredicate
DoublePredicate p3 = a -> true;
boolean x3 = p3.test(3.14);
System.out.println(x3);

//4. IntPredicate
IntPredicate p4 = a -> false;
boolean x4 = p4.test(100);
System.out.println(x4);

//5. LongPredicate
LongPredicate p5 = a -> false;
boolean x5 = p5.test(100);
System.out.println(x5);
```

Predicate 계열의 연결

* `and()` , `or()` , `negate()` 정적 메서드
`Predicate` 함수적 인터페이스 `and()`, `or()`, `negate()`라는 default 메서드를 가지고 있다.
이들은 각각 논리 연산자 `and`, `or`, `!` 와 대응된다.

```
IntPredicate x = a -> a%2==0;
IntPredicate x2 = a -> a%3==0;

//1. and()
IntPredicate x3 = x.and(x2);
boolean result = x3.test(6);
System.out.println(result);

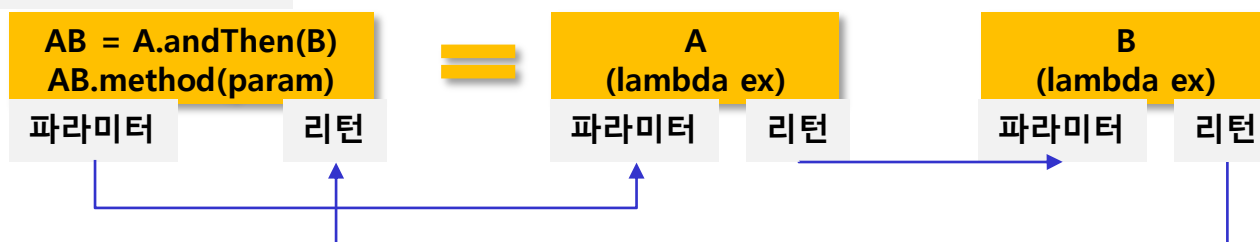
//2. or()
IntPredicate x4 = x.or(x2);
boolean result2 = x4.test(11);
System.out.println(result2);

//3. negate()
IntPredicate x5 = x.negate();
boolean result3 = x5.test(12);
System.out.println(result3);
```

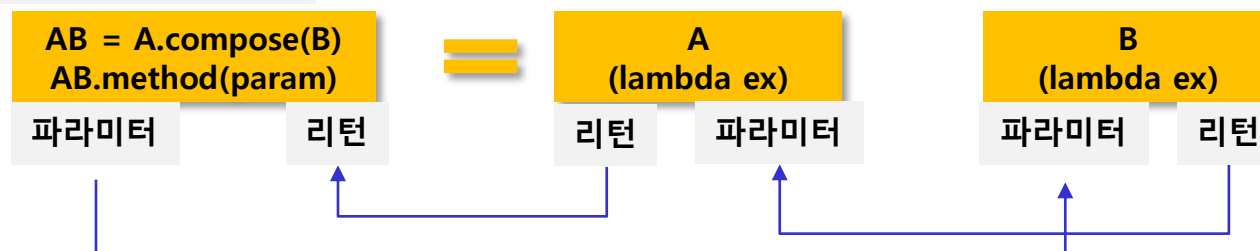
andThen() 과 compose()

- Consumer, Function 계열등에서 두 개의 functional interface를 순차적으로 연결
- 첫 번째 연산 결과를 두 번째의 파라미터로 제공하며, andThen()과 compose() 차이는 적용순서이다.

andThen 처리



compose 처리



1) Consumer 계열의 연결에서 andThen 동작

```
DoubleConsumer con1 = num -> System.out.println(Math.pow(num, 2));  
DoubleConsumer con2 = num -> System.out.println(num + num);
```

```
DoubleConsumer andThen = con1.andThen(con2);  
andThen.accept(10);
```



2) Function 계열의 연결에서 andThen 동작

```
Function<String, String> func1 = name -> "Hello " + name;  
Function<String, String> func2 = data -> "returned : "+data;
```

```
Function<String, String> func3 = func1.andThen(func2);  
System.out.println(func3.apply("홍길동"));
```



func1의 리턴 값이
func2의 파라미터로 사용

```
Function<String, String> func4 = func1.compose(func2);  
System.out.println(func4.apply("장길산"));
```



func2의 리턴 값이
func1의 파라미터로 사용

11. 메서드 참조(method reference)

JDK 21

- 메서드 참조(method reference)는 람다(lambda)표현식에서 불필요한 파라미터 변수를 제거하고 간편하게 사용할 수 있도록 지원한다.
- '참조'라는 단어에서 알 수 있듯이 built-in 메서드를 람다(lambda)로 사용할 수 있도록 참조하는 역할을 담당한다 .
- 즉, 일반 메서드를 람다(lambda) 형태로 사용할 수 있도록 지원하며 메서드를 호출(실행)하는 것이 아니라 참조만 하기 때문에 소괄호는 지정하지 않는다.

메서드 참조	람다(lambda) 표현식
StaticMethod::isPrime	n -> StaticMethod.isPrime(n)
String::toUpperCase	(String w) -> w.toUpperCase()
String::compareTo	(String s, String t) -> s.compareTo(t)
System.out::println	x -> System.out.println(x)
Double::new	n -> new Double(n)
String[]::new	(int n) -> new String[n]

14장. Stream API

1. 스트림 개요 및 특징
2. 스트림 데이터 처리 흐름
3. Stream 사용 3단계
4. 필터링, 매핑, 집계 기능
5. Optional 클래스 및 병렬처리

개요

- 자바 8에 추가된 기능으로서 컬렉션(배열포함)의 저장요소 참조
- 랴다(lambda)식으로 처리할 수 있도록 내부 반복자 제공
- 데이터의 저장이 아닌 처리에 집중

특징

- 병렬 처리
 - 동일한 코드로 순차 또는 병렬 연산을 쉽게 처리
 - 쓰레드에 안전하지 않은 컬렉션도 병렬처리 지원
 - 단, 스트림 연산 중 데이터 원본을 변경하면 안됨
- 지연 연산
 - 스트림을 반환하는 필터-맵 API는 기본적으로 지연(lazy) 연산
 - 지연 연산을 통한 성능 최적화(무상태 중개 연산 및 반복 작업 최소화)
- 반복의 내재화
 - 반복 구조 캡슐화(제어 흐름 추상화, 제어의 역전)
 - 최적화와 알고리즘 분리

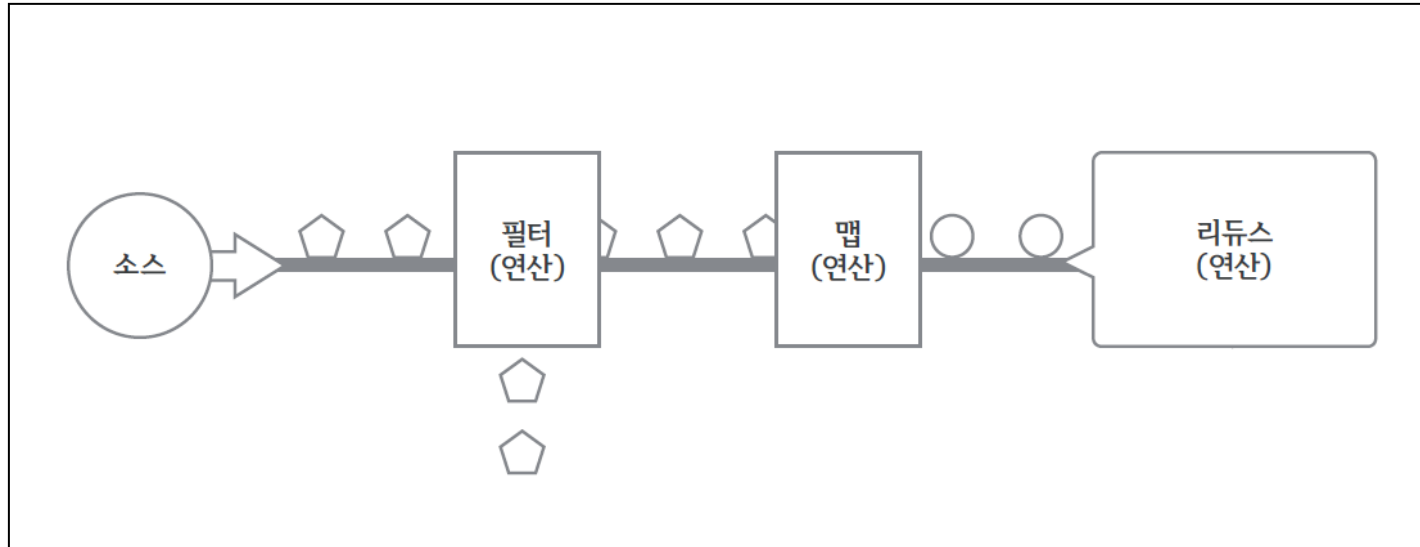
컬렉션

- 외부반복
- 반복을 통해 생성
- 요소의 직접적인 처리
- 유한 데이터 사용
- 재사용 가능

스트림

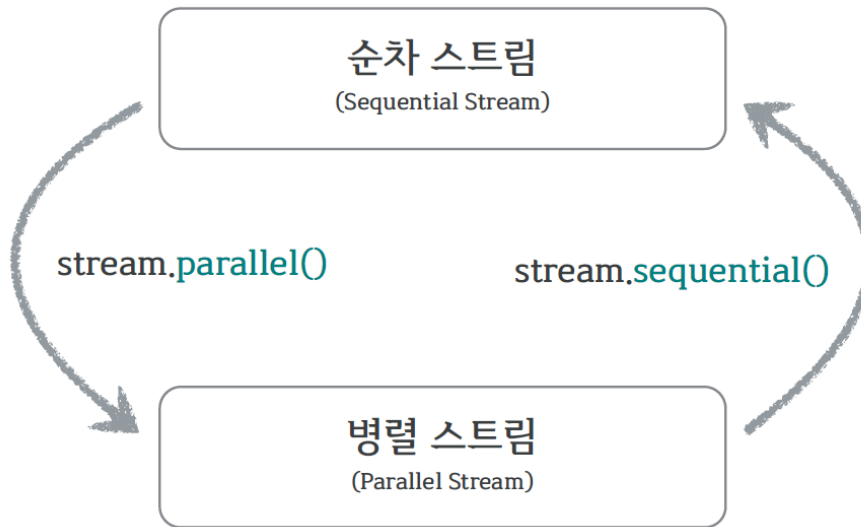
- 내부반복
- 반복을 통해 연산
- 파이프-필터 기반 API
- 무한 연속 구조
- 재사용불가

Stream의 데이터 처리 흐름



- 자바 8부터 새로 추가됨
 - `java.util.stream` 패키지에 스트림(stream) API 존재
 - `BaseStream`
 - 모든 스트림에서 사용 가능한 공통 메서드 제공하는 인터페이스
 - `Stream`
 - 객체 요소 처리
 - 기본 데이터 타입인 `int`, `long`, `double` 요소 처리용 스트림 따로 제공
 - `IntStream`
 - `LongStream`
 - `DoubleStream`
- **Stream 메서드 대부분은 함수형 인터페이스를 인자로 받으며 함수형 인터페이스는 람다(lambda) 표현식을 사용한다.**

- 순차 스트림과 병렬 스트림으로 구분



1단계: 스트림 생성

2단계: 중간 처리

3단계: 최종 연산

```
orders.stream().map(n->n.price).sum();
```

스트림 생성

중개 연산
(스트림 변환)

최종 연산
(스트림 사용)

1단계: 스트림 생성

생성 메서드

- streams(), parallelStream()
- range(...), rangeClosed(...)
- of()
- iterate(...), generate(...)
- lines()

샘플

```
List<String> collection = new ArrayList<>();
Stream<String> stream1 = collection.stream();

// 배열을 스트림으로 변환
int [] numbers= {10, 9, 8, 17, 16, 18, 33};
IntStream stream2 = Arrays.stream(numbers);

// of 메소드로 스트림을 생성
Stream<String> stream3 = Stream.of("홍길동", "김길동", "박길동", "이산");

// 1부터 10까지 유한 스트림 생성
LongStream stream4 = LongStream.rangeClosed (1, 10);
```

2단계: 중간 처리

- 스트림을 받아서 스트림을 반환 (스트림 파이프라인 구성)
- 무상태 연산과 내부 상태 유지 연산으로 나뉨
- 기본적으로 지연(lazy) 연산 처리(성능 최적화)
- 중간 스트림이 생성될 때 바로 중간 처리되는 것이 아니고 최종처리가 시작되기 전까지 중간처리는 지연된다.
최종처리가 시작되면 비로소 컬렉션 요소가 하나씩 중간 스트림에서 처리되고 최종 처리까지 오게 된다.

중간 처리 메서드

종류		리턴 타입	메소드(매개 변수)	소속된 인터페이스
중간 처리	필터링	Stream IntStream LongStream DoubleStream	distinct()	공통
	매핑		filter(...)	공통
			flatMap(...)	공통
			flatMapToDouble(...)	Stream
			flatMapToInt(...)	Stream
			flatMapToLong(...)	Stream
			map(...)	공통
			mapToDouble(...)	Stream, IntStream, LongStream
			mapToInt(...)	Stream, LongStream, DoubleStream
			mapToLong(...)	Stream, IntStream, DoubleStream
			mapToObj(...)	IntStream, LongStream, DoubleStream
			asDoubleStream()	IntStream, LongStream
			asLongStream()	IntStream
			boxed()	IntStream, LongStream, DoubleStream
	정렬		sorted(...)	공통
	루핑	peek(...)	공통	

샘플

```
public class OperatorStreamExam {  
  
    public static void main(String[] args) {  
        // of 메소드로 스트림을 생성  
        //"김" 으로 시작하는 문자열 필터링  
        //필터링된 문자열에 "님" 추가  
        Stream<String> stream = Stream.of("홍길동", "김길동", "박길동", "김길순", "김영희");  
        String [] names=stream  
            .filter(name -> name.startsWith("김"))  
            .map(name -> name+"님")  
            .toArray(String[]::new);  
        for(String name : names) {  
            System.out.println("이름 : "+name);  
        }  
    }  
}
```

3단계: 최종 처리

- 스트림의 요소들을 연산하여 스트림이 아닌 결과를 리턴
- 최종 연산 시 이전의 모든 중간 연산 수행(반복 최소화)
- 작업이 완료되면 더 이상 스트림을 사용할 수 없음

최종 처리 메서드

종류		리턴 타입	메소드(매개 변수)	소속된 인터페이스
최종 처리	매칭	boolean	allMatch(...)	공통
		boolean	anyMatch(...)	공통
		boolean	noneMatch(...)	공통
	집계	long	count()	공통
		OptionalXXX	findFirst()	공통
		OptionalXXX	max(...)	공통
		OptionalXXX	min(...)	공통
		OptionalDouble	average()	IntStream, LongStream, DoubleStream
		OptionalXXX	reduce(...)	공통
		int, long, double	sum()	IntStream, LongStream, DoubleStream
	루핑	void	forEach(...)	공통
	수집	R	collect(...)	공통

샘플

```
public class TerminalStreamExam {  
  
    public static void main(String[] args) {  
  
        IntStream stream = IntStream.of(78, 90, 88, 65, 50, 100);  
        OptionalDouble average=stream.average();  
        System.out.println("평균 : "+average.getAsDouble());  
    }  
}
```

- 중간 처리 기능으로 요소를 걸러내는 스트림 공통 메서드
- 필터링 메서드: `distinct()`, `filter()`

리턴 타입	메소드(매개 변수)	설명
Stream IntStream LongStream DoubleStream	<code>distinct()</code>	중복 제거
	<code>filter(Predicate)</code>	조건 필터링
	<code>filter(IntPredicate)</code>	
	<code>filter(LongPredicate)</code>	
	<code>filter(DoublePredicate)</code>	

- 중간 처리 기능으로 특정 객체에서 특정 데이터를 선택하는 기능 (SQL의 projection 기능)
- flatMapXxx() 메서드 : 스트림의 각 값을 다른 스트림으로 만든 다음에 모든 스트림을 하나의 스트림으로 연결하는 기능. (스트림 평면화)
- mapXxx() 메서드: 입력 데이터를 가공하여 출력 데이터로 반환 기능.

flatMap 메서드

리턴 타입	메소드(매개 변수)	요소 → 대체 요소
Stream<R>	flatMap(Function<T, Stream< R>>)	T → Stream<R>
DoubleStream	flatMap(DoubleFunction<DoubleStream>)	double → DoubleStream
IntStream	flatMap(IntFunction<IntStream>)	int → IntStream
LongStream	flatMap(LongFunction<LongStream>)	long → LongStream
DoubleStream	flatMapToDouble(Function<T, DoubleStream>)	T → DoubleStream
IntStream	flatMapToInt(Function<T, IntStream>)	T → IntStream
LongStream	flatMapToLong(Function<T, LongStream>)	T → LongStream

map 메서드

리턴 타입	메소드(매개 변수)	요소 → 대체 요소
Stream<R>	map(Function<T, R>)	T → R
DoubleStream	mapToDouble(ToDoubleFunction<T>)	T → double
IntStream	mapToInt(ToIntFunction<T>)	T → int
LongStream	mapToLong(ToLongFunction<T>)	T → long
DoubleStream	map(DoubleUnaryOperator)	double → double
IntStream	mapToInt(DoubleToIntFunction)	double → int
LongStream	mapToLong(DoubleToLongFunction)	double → long
Stream<U>	mapToObj(DoubleFunction<U>)	double → U
IntStream	map(IntUnaryOperator mapper)	int → int
DoubleStream	mapToDouble(IntToDoubleFunction)	int → double
LongStream	mapToLong(IntToLongFunction mapper)	int → long
Stream<U>	mapToObj(IntFunction<U>)	int → U
LongStream	map(LongUnaryOperator)	long → long
DobleStream	mapToDouble(LongToDoubleFunction)	long → double
IntStream	mapToInt(LongToIntFunction)	long → Int
Stream<U>	mapToObj(LongFunction<U>)	long → U

- 요소 전체를 반복 및 수집하는 것

peek()

- 중간 처리 메소드
- 중간 처리 단계에서 전체 요소를 루핑하며 추가 작업 하기 위해 사용
- 최종처리 메소드가 실행되지 않으면 지연
- 반드시 최종 처리 메소드가 호출되어야 동작

foreach()

- 최종 처리 메소드
- 파이프라인 마지막에 루핑하며 요소를 하나씩 처리
- 요소를 소비하는 최종 처리 메소드
- sum()과 같은 다른 최종 메소드 호출 불가

- 최종 처리 기능으로 요소들을 처리해 카운팅, 합계, 평균값, 최대값, 최소값 등과 같이 하나의 값으로 산출하는 것
- 대량의 데이터를 가공해서 축소하는 리덕션(Reduction)

집계 메서드

리턴 타입	메소드(매개 변수)	설명
long	count()	요소 개수
OptionalXXX	findFirst()	첫 번째 요소
Optional<T> OptionalXXX	max(Comparator<T>) max()	최대 요소
Optional<T> OptionalXXX	min(Comparator<T>) min()	최소 요소
OptionalDouble	average()	요소 평균
int, long, double	sum()	요소 총합

- `sum( )`, `average( )`, `count( )`, `max( )`, `min( )`
 - 기본 집계 메소드 이용
- `reduce( )` 메소드
 - 프로그래밍화해서 다양한 집계 결과물을 만들 수 있도록 제공 (값으로 반환됨)

인터페이스	리턴 타입	메소드(매개 변수)
Stream	Optional<T>	<code>reduce(BinaryOperator<T> accumulator)</code>
	T	<code>reduce(T identity, BinaryOperator<T> accumulator)</code>
IntStream	OptionalInt	<code>reduce(IntBinaryOperator op)</code>
	int	<code>reduce(int identity, IntBinaryOperator op)</code>
LongStream	OptionalLong	<code>reduce(LongBinaryOperator op)</code>
	long	<code>reduce(long identity, LongBinaryOperator op)</code>
DoubleStream	OptionalDouble	<code>reduce(DoubleBinaryOperator op)</code>
	double	<code>reduce(double identity, DoubleBinaryOperator op)</code>

- 자바8 에서 추가된 클래스
- 연산 후 반환값의 존재가 불확실할 때 사용
- 반환값이 null인 경우에도 안전함.
- Optional, OptionalDouble, OptionalInt, OptionalLong 클래스
- 저장하는 값의 타입만 다를 뿐 제공하는 기능은 거의 동일
- 단순히 집계 값만 저장하는 것이 아님
- 집계 값이 존재하지 않을 경우 디폴트 값을 설정 가능
- 집계 값을 처리하는 Consumer도 등록 가능

Optional 메서드

리턴 타입	메소드(매개 변수)	설명
boolean	isPresent()	값이 저장되어 있는지 여부
T double int long	orElse(T) orElse(double) orElse(int) orElse(long)	값이 저장되어 있지 않을 경우 디폴트 값 지정
void	ifPresent(Consumer) ifPresent(DoubleConsumer) ifPresent(IntConsumer) ifPresent(LongConsumer)	값이 저장되어 있을 경우 Consumer에서 처리

12. 스트림 데이터 수집 (collect)

JDK 21

collect 메서드

- 스트림의 최종 연산처리 함수로서 스트림의 요소를 소비해서 최종 결과를 반환
- `stream.collect(Collectors.메서드)` 형식

Collectors API

- Collectors API은 스트림의 요소를 어떤 식으로 도출할지를 지정한다.
- 3가지로 구성

가. 스트림 요소를 하나의 값으로 리듀스하여 반환

예> `toList(), toSet(), toMap(), counting(), summingInt(), minBy(), maxBy(), summarizingInt(), joining(), reducing()`

나. 요소 그룹화하여 반환

예> `groupingBy()`

다. 요소 분할하여 반환

예> `partitioningBy()`

병렬성(Parallel)

- 멀티 코어 CPU 환경에서 쓰임
- 하나의 작업을 분할해서 각각의 코어가 병렬적 처리하는 것
- 병렬 처리의 목적은 작업 처리 시간을 줄이기 위한 것
- 자바 8부터 요소를 병렬 처리할 수 있도록 하기 위해 병렬 스트림 제공

동시성(Concurrency)

- 동시성 - 멀티 작업을 위해 멀티 스레드가 번갈아 가며 실행하는 성질
- 싱글 코어 CPU를 이용한 멀티 작업
- 병렬적으로 실행되는 것처럼 보임
- 실체는 번갈아 가며 실행하는 동시성 작업

13. 병렬 처리 (Parallel Operation)

JDK 21

- 전체 데이터를 쪼개어 서브 데이터들로 만든 뒤 병렬 처리해 작업을 빨리 끝내는 것
- 자바 8부터 지원하는 병렬 스트림은 데이터 병렬성을 구현한 것
- 멀티 코어의 수만큼 대용량 요소를 서브 요소들로 나누고
- 각각의 서브 요소들을 분리된 스레드에서 병렬 처리

ex) 쿼드 코어(Quad Core) CPU일 경우 4개의 서브 요소들로 나누고 4개의 스레드가 각각의 서브 요소들을 병렬 처리

인터페이스	리턴 타입	메소드(매개 변수)
java.util.Collection	Stream	parallelStream()
java.util.Stream.Stream	Stream	parallel()
java.util.Stream.IntStream	IntStream	
java.util.Stream.LongStream	LongStream	
java.util.Stream.DoubleStream	DoubleStream	



감사합니다.
